People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research

( )
École nationale Supérieure d'Informatique
ex. INI (Institut National de formation en Informatique)

Final Year Project Report

In partial fulfillment of the requirements for the State Engineer Degree in Computer Science

Option: Intelligent Systems and Data (SD)

State of the Art on Transformer Pruning

Presented by:
MAHRAZ ABDELRAHMEN
REZIG HAMAZA

Supervised by:
MEZIANI Lila
GHORAB SARA

Host organization : LCSI ESI

Academic Year: 2025/2026

Abstract

The standard NLP architecture, transformers, is a successful approach to many different tasks. However, the size of the such model is costly to deploy. All parameters, heads, feed-forward neurons, and layers are stored and calculated in a dense model, even if some of these calculations are relatively meaningless for the final output given a specific task. The goal of pruning is to eliminate some of this computation but still allow for usable results. This report will investigate the definitions, scoring and application of pruning to transformers.

We begin with the simple formulation of pruning as a constrained compression problem and discuss the various criteria used to determine what can be removed, distinguish between unstructured sparsity and semi/structured pruning and different pruning regimes. In particular for Transformers, we are concerned with pruning what is possible to actually remove: the weights themselves, attention heads, feed-forward channels, or even entire layers.

However, one persistent drawback can be observed in the survey: the majority of techniques assign individual scores to the units of the model, before eliminating the lowest scoring ones. While this is straightforward, and often performs very well at moderate compression rates, it fails to properly address the more challenging problem of distributing a very small remaining budget over the entire model. At high compression ratios, the layer assignments can be crucial and are more important than the individual score of any unit. In conclusion, this deficiency is presented as the fundamental motivation behind search-based pruning methods.

Keywords: neural network pruning, Transformer compression, structured pruning, saliency criteria, search-based pruning

Résumé

Les modèles Transformer sont devenus des outils courants en traitement automatique du langage, mais leur taille rend leur déploiement coûteux. Un modèle dense stocke et exécute tous ses paramètres, toutes ses têtes d'attention, tous ses neurones feed-forward et toutes ses couches, même lorsqu'une partie de ce calcul contribue peu à la prédiction finale. L'élagage cherche à supprimer une partie de ce calcul tout en gardant un modèle exploitable. Ce rapport étudie la manière dont l'élagage a été formulé, évalué et appliqué aux modèles Transformer.

Le rapport commence par la formulation de l'élagage comme un problème de compression sous contrainte, puis passe en revue les principaux critères utilisés pour décider ce qui peut être retiré. Il présente les méthodes fondées sur la magnitude et les activations, les critères par gradient et développement de Taylor, ainsi que les approches du second ordre comme Optimal Brain Damage et Optimal Brain Surgeon. Il distingue aussi la parcimonie non structurée, semi-structurée et structurée, car ces choix n'ont pas les mêmes effets sur le matériel. Pour les Transformers, l'analyse se concentre sur les unités que l'on peut vraiment supprimer : les poids, les têtes d'attention, les canaux feed-forward et parfois des couches entières.

La revue fait apparaître une limite récurrente. Beaucoup de méthodes attribuent un score local aux unités, puis suppriment celles qui semblent les moins importantes. Cette stratégie reste utile à compression modérée, mais elle répond mal à une question plus difficile : comment répartir un budget très réduit sur l'ensemble du modèle ? À forte compression, la répartition finale entre les couches peut compter davantage que le score isolé d'une seule unité. Le rapport termine donc en présentant cette limite comme la principale motivation des méthodes d'élagage fondées sur la recherche.

Mots-clés : élagage de réseaux de neurones, compression de Transformers, élagage structuré, critères de saillance, élagage par recherche

Arabic Summary

Dedication

Acknowledgements

Contents

List of Figures

List of Tables

List of Acronyms and Abbreviations

Adam	Adaptive Moment Estimation
AE	Autoencoder
BERT	Bidirectional Encoder Representations from Transformers
BF16	Brain Float 16
CE	Cross-Entropy Loss
CNN	Convolutional Neural Network
CoFi	Coarse-to-Fine Structured Pruning via Learned Masks
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DarwinLM	Training-Aware Evolutionary Structured Pruning
ELU	Exponential Linear Unit
EMA	Exponential Moving Average
EvoPress	Evolutionary Search over Layer-Wise Compression Profiles
FFN	Feed-Forward Network
FLOPs	Floating Point Operations
FP16	16-bit Floating Point
FP32	32-bit Floating Point
FT	Fine-Tuning
GA	Genetic Algorithm
GELU	Gaussian Error Linear Unit
GLUE	General Language Understanding Evaluation

GPT	Generative Pre-trained Transformer
GPU	Graphics Processing Unit
INT4	4-bit Integer
INT8	8-bit Integer
KD	Knowledge Distillation
KL	Kullback-Leibler Divergence
LLaMA	Large Language Model Meta AI
LLM	Large Language Model
LDLQ	LDL-Transpose Quantization
LLM-Pruner	Dependency-Aware Gradient-Based Structured Pruning
LoRA	Low-Rank Adaptation
MACs	Multiply-Accumulate Operations
MAE	Mean Absolute Error
MHA	Multi-Head Attention
MLP	Multi-Layer Perceptron
MNLI	Multi-Genre Natural Language Inference
MOP	Multi-Objective Optimization Problem
MRP	Multi-Weight Removal Process
MSE	Mean Squared Error
N:M	Semi-Structured Sparsity Pattern
NAS	Neural Architecture Search
NLP	Natural Language Processing
NSGA-II	Non-Dominated Sorting Genetic Algorithm II
OBS	Optimal Brain Surgeon
OPT	Open Pre-trained Transformer
PPL	Perplexity
QNLI	Question Natural Language Inference

ReLU	Rectified Linear Unit
RMSNorm	Root Mean Square Normalization
RNN	Recurrent Neural Network
SGD	Stochastic Gradient Descent
SparseGPT	Second-Order Unstructured Pruning via Hessian Compensation
SQuAD	Stanford Question Answering Dataset
SVD	Singular Value Decomposition
Tanh	Hyperbolic Tangent
Wanda	Weight and Activation-Based Pruning Criterion
ZipLM	Runtime-Aware Second-Order Structured Pruning

General Introduction

The rapid evolution of natural language processing over the past decade is largely driven by the Transformer architecture. Its ability to model complex, long-range dependencies in parallel has led to state-of-the-art performance across nearly all sequence modeling tasks. However, this expressive power comes at a severe computational cost. Modern Large Language Models (LLMs) require massive amounts of memory, immense computational throughput, and high energy consumption. These requirements create a strict bottleneck, making the deployment of these models in production environments or on resource-constrained edge devices highly impractical without significant compression.

To bridge the gap between training-time scale and deployment-time feasibility, neural network pruning has emerged as a premier compression strategy. The fundamental objective of pruning is to identify and remove redundant parameters or entire computational units from a trained network while preserving its original predictive behavior. Yet, pruning is a multifaceted optimization problem. A practitioner must decide the topology of the pruning (whether to scatter zeros arbitrarily or remove entire structural blocks), the criterion for removal (relying on weight magnitude, gradients, or loss curvature), and the recovery regime required to heal the network post-compression.

As the literature on Transformer pruning has expanded, a critical limitation has become apparent across the state-of-the-art. The vast majority of current methods rely on local scoring, which means evaluating the importance of a single weight, attention head, or layer in isolation before the joint effect of all removals is known. While local criteria are highly effective at moderate compression levels, they degrade rapidly at high sparsity. At aggressive compression ratios, the global allocation of the sparsity budget across the model's layers becomes vastly more important than the precision of any individual local score. Evaluating local criteria without considering global structural interactions limits the true potential of Transformer compression.

The objective of this report is to review comprehensively the state-of-the-art in Transformer pruning, analyze the existing methodologies, and formally isolate this "allocation problem" to motivate future search-based pruning frameworks.

To achieve this, the report is divided into two main parts. The first part establishes the foundational knowledge required for this study and spans two chapters.Chapter 1 covers the theoretical foundations of neural networks. Chapter 2 then examines the architecture of the Transformer, defines its structural components and the specific hardware-aware efficiency metrics that practically drive the need for compression.

The second part surveys the extensive literature on Transformer pruning and evaluates the state of the art. It begins with Chapter 3, which formalizes the pruning problem mathematically by categorizing the different structural regimes. Chapter 4 reviews Unstructured and Heterogeneous Search-based methods, focusing on techniques that prioritize theoretical flexibility and search space exploration at the weight and layer-profile levels. Chapter 5 then shifts the focus to Structured Methods, detailing algorithms that excise complete architectural units to achieve hardware-agnostic, dense-shape acceleration. Following this review, Chapter 6 synthesizes the literature through a cross-regime empirical comparison. It highlights the critical limitations in current evaluation protocols and formally defines the global allocation problem that local scoring methods fail to solve. Finally, Chapter 7 concludes the report, summarizing the core findings and establishing the clear motivation for treating future model compression not as a local ranking task, but as a global configuration search.

Chapter 1

Foundations

1.1 Introduction

The objective of pruning is to preserve the behaviour of a trained neural network while changing its size by removing some of its parameters or larger components. In order to investigate what is eligible to be removed, and how the network will react to the removal of certain components, it's important first to describe the objects which pruning can operate on: Neurons, weights, activations, layers, feed-forward neural network. The vocabulary is therefore not presented only as general knowledge.

We start this chapter by explaining what a single artificial neuron looks like . This is important for pruning as the weights and biases are the lowest parameters available to train, while the activation dictates how the data continues its passage once these weights and biases have been modified. From this, we naturally arrive at multi-layer perceptrons, loss functions, gradient-based optimization and backpropagation, as we will evaluate the quality of a pruning procedure by how much the output behavior or loss is altered when removing a section of a network Goodfellow et al., 2016; Rumelhart et al., 1986.

1.2 Parameterization of Neural Networks

The artificial neuron is a computational unit that takes a vector of inputs, applies a linear transformation, and then passes the result through a non-linear activation function. This structure allows neural networks to model complex, non-linear relationships in data. The linear part of the neuron is often referred to as an affine transformation, which combines a weighted sum of the inputs with a bias term. The non-linear activation function is crucial for enabling the network to learn and represent complex patterns, as it allows the model to capture non-linearities in the data. Common activation functions include ReLU, sigmoid, and tanh, each with its own properties and use cases.

1.2.1 The Neuron as a Linear Model

The artificial neuron, formally introduced by McCulloch and Pitts, 1943, serves as the fundamental building block of neural networks. Mathematically, it performs an **affine transformation** on its inputs followed by a non-linear activation f which is represented in Figure 1.1 and expressed in Eq.(1.1).

$$z = \boldsymbol{w}^T \boldsymbol{x} + b = \sum_{i=1}^{n} w_i x_i + b \tag{1.1}$$

Given an input $\boldsymbol{x} \in \mathbb{R}^n$, weights $\boldsymbol{w} \in \mathbb{R}^n$ scale the importance of each feature, while the bias $b \in \mathbb{R}$ acts as an offset. Figure 1.1 shows this affine unit before the activation is applied: the weighted inputs are summed with the bias to produce the pre-activation response z.

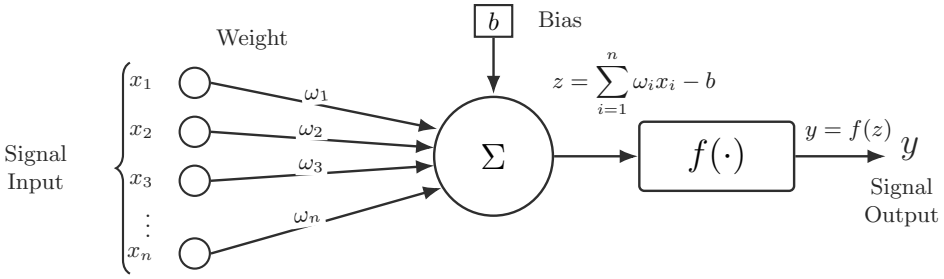


Figure 1.1: A single artificial neuron with input vector $\boldsymbol{x}$, weight vector $\boldsymbol{w}$, bias b, and pre-activation output z. The activation function f is applied to z to produce the final output y.

The term $\boldsymbol{w}^T \boldsymbol{x}$ measures how strongly the input matches the learned weights. This affine computation is the basic operation repeated across layers in larger neural networks.

1.2.2 Activation Functions

The neural network's ability to model complex patterns depends on the non-linear activation functions applied after each affine transformation. If we were to stack multiple layers without any non-linearity, the entire network would collapse into a single affine transformation.

Therefore, depth alone is not enough: neural networks require non-linear **activation functions** to model complex patterns Goodfellow et al., 2016.

As shown in Figure 1.1, the activation function f takes the pre-activation output z and produces the final output $y = f(z)$. The choice of activation function can significantly affect the network's performance, convergence, and ability to learn from data.

Classical Saturating Activations

Sigmoid (Eq.(1.2)) and hyperbolic tangent (Eq.(1.3)) functions were often used in early networks. While these functions take real-valued inputs and output bounded values, which could be useful in the case of probabilities or normalized hidden states, they have very small derivatives when the input is very large or very small. This saturation causes learning to be very slow in deep networks Glorot and Bengio, 2010; Goodfellow et al., 2016.

$$\sigma(x) = \frac{1}{1+e^{-x}}, \qquad \sigma : \mathbb{R} \to (0,1). \tag{1.2}$$

$$\tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}, \qquad \tanh : \mathbb{R} \to (-1,1). \tag{1.3}$$

Rectified Activations

Rectified Linear Units (ReLU) became common in deep learning because they are simple, computationally cheap, and reduce saturation for positive inputs Nair and Hinton, 2010. ReLU is defined as showing in Eq.(1.4):

$$\mathrm{ReLU}(x) = \max(0,x), \qquad \mathrm{ReLU} : \mathbb{R} \to [0,\infty). \tag{1.4}$$

Leaky ReLU keeps a small non-zero slope for negative inputs, reducing the risk that a unit becomes permanently inactive Maas et al., 2013, as shown in Eq.(1.5):

$$\mathrm{LeakyReLU}(x) = \begin{cases} x, & x \geq 0, \\ \alpha x, & x < 0, \end{cases} \qquad \alpha \in (0,1), \quad \mathrm{LeakyReLU} : \mathbb{R} \to \mathbb{R}. \tag{1.5}$$

The Exponential Linear Unit (ELU) uses a smooth negative branch and a linear positive branch Clevert et al., 2016, which can lead to faster convergence and improved performance in some cases, as shown in Eq.(1.6):

$$\mathrm{ELU}(x) = \begin{cases} x, & x \geq 0, \\ \alpha(e^{x}-1), & x < 0, \end{cases} \qquad \alpha > 0, \quad \mathrm{ELU} : \mathbb{R} \to (-\alpha,\infty). \tag{1.6}$$

Modern Transformer architectures often use the Gaussian Error Linear Unit (GELU), a smooth activation that weights inputs by their value under the standard Gaussian cumulative

distribution Hendrycks and Gimpel, 2016, as shown in Eq.(1.7):

$$\mathrm{GELU}(x) = x\Phi(x) \approx \frac{x}{2}\left(1+\tanh\left(\sqrt{\frac{2}{\pi}}(x+0.044715x^3)\right)\right). \tag{1.7}$$

Figure 1.2 compares the shapes of these activation functions on the same input range. The plot makes the difference between saturating functions and rectified functions visible.

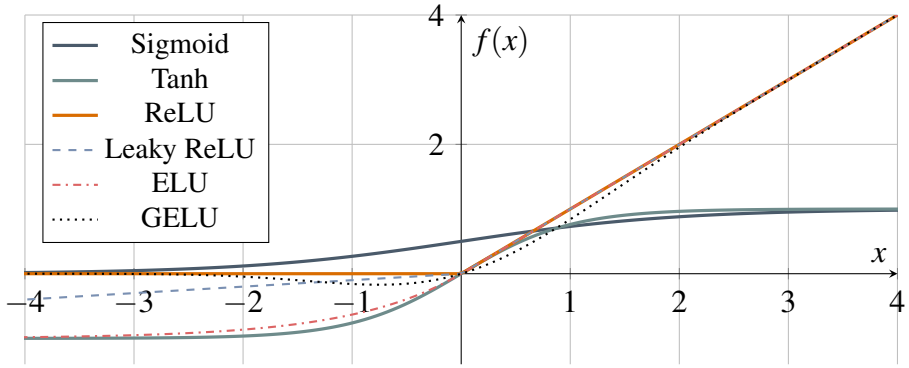


Figure 1.2: Comparison of common activation functions

1.2.3 The Multi-Layer Perceptron (MLP)

A single activated neuron can transform an input into a simple response, but practical models require many such units to work together. A Multi-Layer Perceptron extends the neuron model by organizing neurons into successive layers, so that each layer transforms the representation produced by the one before it.

Going from a single neuron to an MLP increases expressiveness by composing affine transformations with non-linear activations across layers Rumelhart et al., 1986. MLPs are *feedforward* and typically *fully connected*, meaning each neuron connects to all neurons in the next layer.

Eq. (1.8) formalizes the computation performed by each layer of an MLP, to transform the raw input $\boldsymbol{x}$ into the final output (prediction)$\boldsymbol{y}$.

$$\begin{aligned} \boldsymbol{h}^{(0)} &= \boldsymbol{x}, \\ \boldsymbol{h}^{(\ell)} &= f(\boldsymbol{W}^{(\ell)}\boldsymbol{h}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}) \quad \text{for } \ell = 1,2,\ldots,L-1, \\ \boldsymbol{y} &= f_{\text{out}}(\boldsymbol{W}^{(L)}\boldsymbol{h}^{(L-1)} + \boldsymbol{b}^{(L)}), \end{aligned} \tag{1.8}$$

Figure 1.3 illustrates a typical MLP with one input layer, two hidden layers, and one output layer.

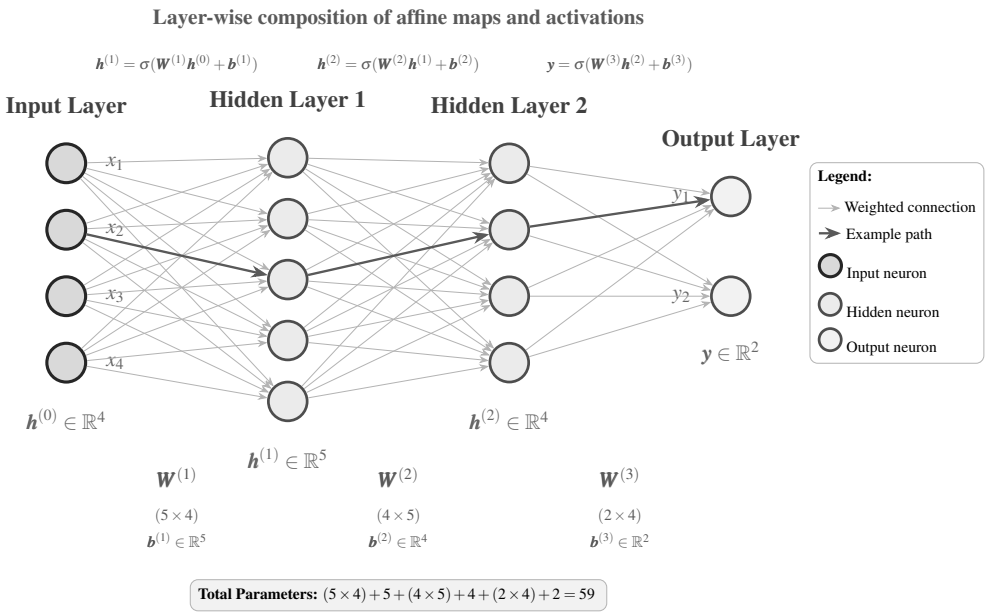


Figure 1.3: Fully connected MLP with 4 input features, two hidden layers (5 and 4 neurons), and 2 output neurons.

1.3 Optimization and Gradients

We outlined how an MLP generates a prediction: by a composition of transformations applied at each layer. With forward computation, learning can be thought of as figuring out what the weights and biases should be given the data. Network construction then becomes an optimization problem: select the parameters that make the output as close as possible to the target on a training set.

Hence, a quantitative measure must be provided that can actually be minimized; this provides the rationale for empirical risk minimization.

1.3.1 Empirical Risk Minimization

Given a dataset $\mathscr{D} = \{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$, learning consists of finding parameters $\boldsymbol{\theta}$ that minimize the discrepancy between the model predictions $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ and the target values. This is commonly formulated as empirical risk minimization Vapnik, 1998:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \hat{R}(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\theta}} \frac{1}{m} \sum_{i=1}^{m} \mathscr{L}(f_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)}), y^{(i)}). \tag{1.9}$$

This empirical-risk objective in Eq. (1.9) gives the general learning template: choose parameters by minimizing the average loss over the training set. It does not specify the loss itself. That choice is determined by the problem and the structure of the output.

Loss Functions

Now we have an optimization problem. But what are we minimizing? A loss function accounts for the discrepancy between predictions and targets and makes explicit assumptions about the problem and the output space. Table 1.1 summarizes the most common loss functions used in neural network training, along with their formulas, target types, and properties. The choice of loss function depends on the task at hand (classification vs regression) and the nature of the targets (hard one-hot labels, soft probability distributions, or continuous real values.).

Table 1.1: Summary of common neural network loss functions.

Loss	Task	Formula	Target type	properties
Cross-Entropy	Classification	$\mathscr{L}_{\text{CE}}(\hat{\boldsymbol{y}}, \boldsymbol{y}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k$	Soft or one-hot	Measures total encoding cost of $\boldsymbol{y}$ under code optimal for $\hat{\boldsymbol{y}}$; reduces to $-\log \hat{y}_c$ for hard labels.
KL Divergence	Classification	$D_{\text{KL}}(\boldsymbol{y} \Vert \hat{\boldsymbol{y}}) = \sum_{k=1}^{K} y_k \log \frac{y_k}{\hat{y}_k}$	Soft distribution	Measures *excess* encoding cost; $D_{\text{KL}} = \mathscr{L}_{\text{CE}} - H(\boldsymbol{y})$. Preferred over CE when targets are soft.
MSE (L2)	Regression	$\mathscr{L}_{\text{MSE}}(\hat{y}, y) = \frac{1}{m}\sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)})^2$	Continuous	Penalises errors quadratically; equivalent to maximum likelihood estimation (MLE) under Gaussian noise. Sensitive to outliers.
MAE (L1)	Regression	$\mathscr{L}_{\text{MAE}}(\hat{y}, y) = \frac{1}{m}\sum_{i=1}^{m} \lvert \hat{y}^{(i)} - y^{(i)} \rvert$	Continuous	Penalises errors linearly; robust to heavy-tailed distributions and outliers.

Notes. K: number of classes; $\hat{\boldsymbol{y}} \in \Delta^K$: predicted probability vector (softmax output); $\boldsymbol{y}$: target distribution (one-hot or soft); c: index of the true class; $H(\boldsymbol{y}) = -\sum_k y_k \log y_k$: entropy of the target; m: number of samples; $\hat{y}^{(i)}, y^{(i)}$: predicted and true scalar outputs for sample i. When $\boldsymbol{y}$ is one-hot, $H(\boldsymbol{y}) = 0$ and $D_{\text{KL}} = \mathscr{L}_{\text{CE}}$.

1.3.2 Algorithms for Gradient-Based Optimization

With the loss function established, the remaining question is how the resulting error signal propagates back through the network to update the parameters.

Training neural networks is usually a problem of finding a parameter vector $\theta \in \mathbb{R}^d$ that minimizes the scalar objective $J(\theta)$ where J is the empirical risk defined by the loss function and the training data. Formally, we want to solve (1.10):

$$\theta^* = \arg\min_{\theta} J(\theta). \tag{1.10}$$

In modern neural networks, the parameter space is high-dimensional and the objective is generally non-convex, so a closed-form solution for the optimal parameters is usually unavailable. Training therefore proceeds by iteratively updating the parameters in directions suggested by the gradient of the loss Bottou et al., 2018.

The gradient $\nabla J(\theta)$ gives the direction of steepest local increase of the objective. Optimization methods use this information in reverse: they adjust the parameters in a direction that

is expected to reduce the loss. This idea gives the basic gradient descent algorithm and also underlies the adaptive optimizers used in current neural-network training.

Gradient Descent

Gradient descent updates parameters in the negative-gradient direction. For iteration t and learning rate $\eta > 0$, the update is formulated in Eq.(1.11):

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t). \tag{1.11}$$

The learning rate controls the step size. If it is too small, optimization may progress slowly; if it is too large, the updates can overshoot useful regions of the objective surface. In large-scale learning, the exact gradient over the full dataset is often replaced by a stochastic estimate computed on a mini-batch. This gives stochastic gradient descent, which is cheaper per update and is widely used for neural-network training Bottou et al., 2018.

Adam

Adam is an adaptive gradient-based optimizer that combines momentum with per-parameter learning-rate adaptation Kingma and Ba, 2015. Instead of using only the current gradient, Adam keeps exponential moving averages of the first and second moments of the gradient, which are updated as shown in Eq.(1.12):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{1.12}$$

where $g_t = \nabla J(\theta_t)$ is the gradient estimate at iteration t, m_t tracks the average direction of recent gradients, and v_t tracks their squared magnitude.

Because both moving averages are initialized at zero, Adam applies bias correction computed as follows Eq.(1.13):

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}. \tag{1.13}$$

The parameter update is then given by (1.14):

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}. \tag{1.14}$$

Adam is commonly used because it is robust to gradient-scale differences across parameters and often requires less manual learning-rate tuning than plain gradient descent. Later variants such as AdamW decouple weight decay from the adaptive update, which is useful in many deep-learning settings Loshchilov and Hutter, 2019.

Gradient descent and Adam describe how parameters should be updated, but they all still presuppose the necessary gradients being available. In deep, multi-layered networks calculating such gradients may be computationally expensive and hence demands a separate procedure called backpropagation.

1.3.3 Backpropagation

Backpropagation computes these gradients efficiently by applying the chain rule from the output layer back through the network Rumelhart et al., 1986. In compact form, it provides the quantities $\nabla_\theta \mathscr{L}$ where θ denotes the trainable parameters and $\mathscr{L}$ is the loss function.

For this pruning-focused report, the detailed derivation is not needed. What matters is that backpropagation links model error to individual parameters and intermediate structures. This link is used later by gradient-based pruning criteria, Taylor approximations, Fisher-based scores, and recovery stages after structural removal.

1.4 Conclusion

This chapter established the fundamental mathematical and optimization frameworks underlying deep neural networks. We defined the core computational units, the affine transformations and non-linear activations, that compose feed-forward networks, and formalized the training process as an empirical risk minimization problem. Furthermore, we introduced the key optimization concepts of gradients and the Hessian matrix, which are central to understanding how networks learn and how pruning algorithms evaluate parameter importance.

While these theoretical principles apply universally to feed-forward networks, modern natural language processing relies on highly specialized architectural designs. To understand how pruning algorithms are applied in practice, these general matrix operations must be mapped to specific structural components. Consequently, the following chapter will focus exclusively on the Transformer architecture, detailing its internal mechanics, parameter distribution, and the severe computational bottlenecks that make its compression an absolute necessity for real-world deployment.

Chapter 2

Transformer Architectures and Efficiency

2.1 Introduction

While Multi-Layer Perceptrons offer a generic learning model over vector inputs, they do not inherently leverage any underlying structure of the data. For this report, the architecture of interest is the Transformer Vaswani et al., 2017. Since its introduction, the Transformer's ability to model long-range dependencies in parallel has made it the foundation of state-of-the-art sequence modeling. However, this expressive power comes at a severe computational cost. To effectively compress such a model, one cannot treat it as a generic sequence of matrices; rather, one must deeply understand its components.

The purpose of this chapter is to dissect the Transformer architecture specifically through the lens of computational cost and structural parameterization. The chapter begins by detailing the core building blocks of the network, namely the Multi-Head Attention mechanism and the position-wise Feed-Forward Networks (FFNs). Special emphasis is placed on the matrix dimensions and structural design of these components, as they dictate the exact locations of the parameters that pruning algorithms will later target.

Building on these components, the chapter then examines the complete Encoder-Decoder architecture before shifting focus to the primary motivation for this report: the computational bottleneck. We analyze the quadratic scaling of self-attention and the massive memory footprints of dense weight matrices. Finally, the chapter introduces hardware-aware efficiency metrics, such as FLOPs, memory bandwidth, and latency.

2.2 Transformer Networks

Transformer networks, introduced by Vaswani et al., 2017, changed sequence modeling by replacing recurrent processing with attention-based computation. Instead of processing tokens one after another, a Transformer can compare tokens across the whole sequence in parallel. This makes it highly effective for language modelling, translation, summarization, and other sequence tasks. The original architecture was an encoder-decoder model for machine translation, while later systems often use encoder-only variants such as BERT Devlin et al., 2019 or decoder-only variants such as LLaMA Touvron et al., 2023.

2.3 Core Architectural Components: Building Blocks of the Transformer

Before separating the model into encoder and decoder stacks, it is useful to define the components repeated inside those stacks. Transformer layers are built from token embeddings, positional information, attention projections, feed-forward channels, residual connections, and normalization layers. These components are also the main targets for compression, because they determine both parameter count and runtime cost.

Input Representation: Embeddings and Positional Encoding

Every token is mapped to a continuous embedding vector. Since self-attention does not inherently encode token order, positional information is added to the token representations. The original Transformer uses sinusoidal positional encodings Vaswani et al., 2017, shown in Eq.(2.1), where d_{model} is the embedding dimension and i indexes the dimensions of the positional encoding:

$$PE_{(\text{pos},2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(\text{pos},2i+1)} = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right). \tag{2.1}$$

Self-Attention Mechanism

Self-attention allows each token representation to aggregate information from the rest of the sequence. The mechanism uses three learned projections of each input: queries, keys, and values Vaswani et al., 2017. Queries represent what a token is looking for, keys represent what each token offers for comparison, and values contain the information that will be aggregated.

For an input sequence representation $\boldsymbol{X} \in \mathbb{R}^{n \times d_{\text{model}}}$, the projections are computed as shown in Eq.(2.2), where $\boldsymbol{W}^Q, \boldsymbol{W}^K, \boldsymbol{W}^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ are learned weight matrices for queries, keys, and values respectively:

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}^Q, \quad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}^K, \quad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}^V. \tag{2.2}$$

Scaled dot-product attention is then computed as shown in Eq.(2.3), where d_k is the dimen-

sion of the queries and keys:

$$\text{Attention}(\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}}\right)\boldsymbol{V}. \tag{2.3}$$

The factor $\sqrt{d_k}$ prevents dot products from becoming too large, which would push the softmax into saturated regions with weak gradients.

Multi-Head Attention

Multi-head attention applies several attention mechanisms in parallel, allowing the model to attend to different representation subspaces and token relationships at the same time Vaswani et al., 2017. Eq.(2.4) defines the computation for each head i, where $\boldsymbol{W}_i^Q, \boldsymbol{W}_i^K, \boldsymbol{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ are the projection matrices for head i:

$$\text{head}_i = \text{Attention}(\boldsymbol{X}\boldsymbol{W}_i^Q, \boldsymbol{X}\boldsymbol{W}_i^K, \boldsymbol{X}\boldsymbol{W}_i^V), \quad i = 1,\ldots,h. \tag{2.4}$$

The head outputs are concatenated and projected back to the model dimension as expressed in Eq.(2.5), where $\boldsymbol{W}^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ is the output projection matrix:

$$\text{MultiHead}(\boldsymbol{X}) = \text{Concat}(\text{head}_1,\ldots,\text{head}_h)\boldsymbol{W}^O. \tag{2.5}$$

Feed-Forward Network

After attention mixes information across tokens, a position-wise Feed-Forward Network (FFN) processes each token representation independently The FFN usually expands the hidden representation to a larger intermediate dimension d_{ff} and then projects it back to d_{model}. In the original Transformer, d_{ff} is typically much larger than d_{model}, making FFN channels a major source of parameters and a natural target for structured pruning.

Residual Connections and Layer Normalization

Each attention or FFN sub-layer is wrapped by a residual path and layer normalization. Residual connections preserve a direct path for information and gradients, while normalization stabilizes the scale of hidden representations. This design is essential for training deep Transformer stacks and is also important when later compression methods remove heads, channels, or blocks Ba et al., 2016; He et al., 2016; Vaswani et al., 2017.

2.3.1 Encoder and Decoder Architecture

With the core components defined, the encoder and decoder can be described cleanly. The full encoder-decoder Transformer contains two main stacks:

- **Encoder:** Processes the complete input sequence in parallel and produces contextual source representations.

- **Decoder:** Generates the output sequence autoregressively while attending to both previous target tokens and the encoder output.

The encoder converts the source sequence into a contextual memory representation $\boldsymbol{H}_{\text{enc}}$. The decoder uses this memory through cross-attention while also using masked self-attention to prevent access to future target tokens. The complete architecture is shown in Figure 2.1.

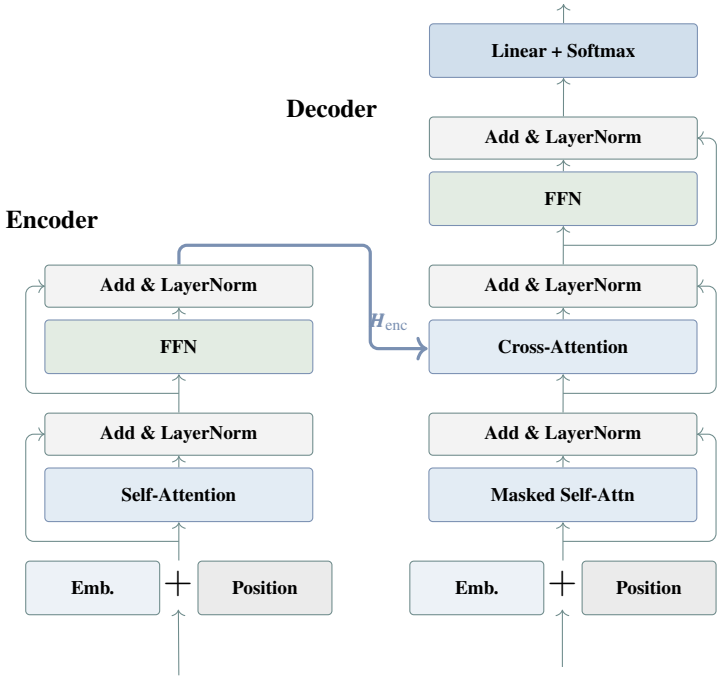


Figure 2.1: Complete Encoder-Decoder Transformer architecture

The Encoder Stack

An encoder layer contains multi-head self-attention followed by a feed-forward network, with residual connections and normalization around each sub-layer. In encoder self-attention, the queries, keys, and values all come from the same source sequence. This makes the encoder bidirectional: every input token can attend to every other input token. Figure 2.2 shows the internal order of these operations before the resulting representation is passed to the decoder. After N encoder layers, the output is the memory matrix $\boldsymbol{H}_{\text{enc}} \in \mathbb{R}^{n \times d_{\text{model}}}$, which summarizes the source sequence for the decoder.

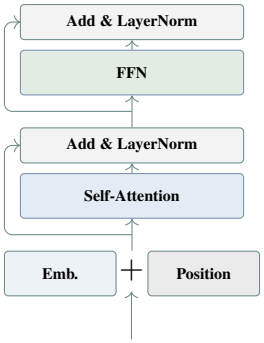


Figure 2.2: Transformer encoder layer architecture

The Decoder Stack

A decoder layer adds masked self-attention and cross-attention before the feed-forward network, enabling autoregressive generation while attending to encoder outputs. Masked self-attention restricts token t to positions $\leq t$, preventing information leakage from future target tokens. Cross-attention then links the decoder to the encoder: decoder states act as queries, while encoder states provide keys and values. Figure 2.3 shows this additional cross-attention block, which is the main structural difference between the decoder layer and the encoder layer.

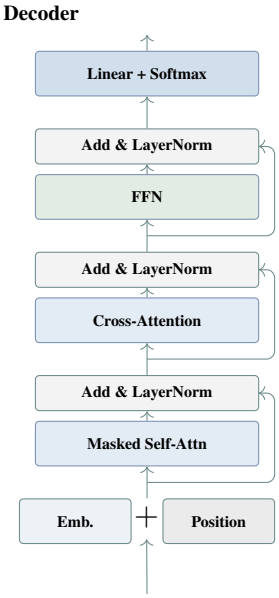


Figure 2.3: Transformer decoder layer architecture

Encoder-Decoder Information Flow

The complete sequence-to-sequence computation proceeds as follows:

1. **Encoding phase.** Source tokens $\boldsymbol{x} = (x_1, \ldots, x_n)$ are embedded, combined with positional information, and passed through N encoder layers. The output is $\boldsymbol{H}_{\text{enc}}$.

2. **Decoding phase.** Target tokens $\boldsymbol{y} = (y_1, \ldots, y_t)$ are embedded and processed autoregressively. Each decoder layer applies masked self-attention, cross-attention over $\boldsymbol{H}_{\text{enc}}$, and a feed-forward transformation.

3. **Output phase.** The final decoder representation is projected to vocabulary logits. A softmax layer converts logits into token probabilities, and the selected token is fed back into the decoder at the next step.

At this point in the report, we can view the Transformer as an arrangement of embedding layers, attention layers, feed-forward functions, and encoder–decoder connections. This architecture accounts for why it performs well on sequence modelling tasks, since each position can refine its representation using information from the rest of the sequence. This expressive structure, however, is computationally demanding. The next point examines where this cost arises inside the Transformer architecture.

2.3.2 The Computational Bottleneck of Transformers

While powerful, the Transformer's success comes at a significant computational cost, which forms a central motivation for the compression techniques explored in this report. The standard Transformer architecture replaces recurrence with self-attention, enabling strong parallelism and high performance across sequence modelling tasks Vaswani et al., 2017. However, this advantage comes with a major bottleneck: in full self-attention, each token attends to every other token in the sequence. For a sequence of length n, this requires an attention score matrix of size $n \times n$, making the attention component scale quadratically with sequence length in both computation and memory Beltagy et al., 2020; Tay et al., 2022.

This quadratic behaviour has direct practical implications. Doubling the sequence length from 512 to 1024 multiplies the attention matrix size by four, since $1024^2/512^2 = 4$. As a result, processing long documents or long-context inputs becomes substantially more expensive, which has motivated a large body of work on efficient Transformer variants and sparse or linear attention mechanisms Beltagy et al., 2020; Tay et al., 2022.

At the same time, modern language models benefit strongly from scaling model size, training data, and compute Kaplan et al., 2020. Later scaling-law work further showed that model size and the number of training tokens should be scaled together under a fixed compute budget, highlighting the continuing importance of large-scale training Hoffmann et al., 2022. This creates a tension between scale and deployment: larger models often achieve better performance, but practical inference is constrained by latency, memory, energy, and hardware availability. This tension motivates compression methods such as pruning, quantization, and knowledge distillation Han et al., 2016; Sanh et al., 2019.

The computational bottleneck defined above can be considered in several different ways. A Transformer may be expensive because of the size of its parameters, the memory required for activations, the number of arithmetic operations it performs, or the time required to run it on a specific hardware platform. Before discussing model efficiency in more detail, it is therefore useful to introduce the most common measures used to assess computational cost.

2.3.3 Hardware-Aware Efficiency Metrics

Compression is evaluated through several distinct efficiency metrics. Parameter count, memory footprint, arithmetic cost, latency, and throughput capture different aspects of model execution. Hardware-aware evaluation therefore requires separate treatment of storage cost, arithmetic cost, and runtime behavior Sze et al., 2017.

Parameter Count and Model Size

Let $\boldsymbol{\theta} = \{W^{(l)}, b^{(l)}\}_{l=1}^{L}$ denote the parameters of a network. The total parameter count is definded by Eq.(2.6):

$$P = \sum_{l=1}^{L} \left(|W^{(l)}| + |b^{(l)}| \right). \tag{2.6}$$

If the parameters are stored with precision b_w bits. Eq.(2.7) gives the model size in bytes:

$$M_{\text{model}} = \frac{P\, b_w}{8} \quad \text{bytes}. \tag{2.7}$$

This follows from counting the stored elements and multiplying by the number of bytes used by each element PyTorch Contributors, 2026a, 2026b. This quantity determines whether a model fits within device memory and how much parameter traffic must be sustained during inference. Quantization reduces b_w, pruning reduces the number of active parameters, and low-rank factorization reduces the parameterization itself.

For sparse models, storage depends on both the remaining weights and the associated metadata. If P_{nz} denotes the number of non-zero weights and M_{meta} the storage needed for indices or masks, the model size is given by (2.8):

$$M_{\text{sparse}} = \frac{P_{\text{nz}}\, b_w}{8} + M_{\text{meta}}. \tag{2.8}$$

MACs, FLOPs, and Arithmetic Intensity

For a dense matrix multiplication $Y = AB$ with $A \in \mathbb{R}^{m\times k}$ and $B \in \mathbb{R}^{k\times n}$, the arithmetic cost is defined by Eq. (2.9):

$$\text{MACs}(A,B) = mkn, \qquad \text{FLOPs}(A,B) \approx 2mkn. \tag{2.9}$$

The distinction is conventional: one multiply-accumulate corresponds to one MAC, while FLOP counting often treats the multiply and the addition as two floating-point operations NVIDIA, 2026b. In Transformer layers, the feed-forward subnetwork contributes large dense matrix products, while self-attention adds the sequence-dependent $O(n^2 d_{\text{model}})$ cost of score computation and value aggregation.

Arithmetic cost alone is insufficient for hardware analysis. A useful complementary quantity is the arithmetic intensity is defined in Eq.(2.10):

$$I = \frac{\text{FLOPs}}{\text{Bytes Moved}}, \tag{2.10}$$

which measures how much computation is performed per byte transferred NVIDIA, 2026b; Williams et al., 2009. Low arithmetic intensity indicates a memory-bound workload, whereas high arithmetic intensity indicates a compute-bound workload.

Bandwidth, Latency, and Throughput

Memory bandwidth measures the rate at which data can be transferred between memory and compute units. If B_{move} bytes are transferred during an operation that takes time T, the effective bandwidth demand is calculated by Eq.(2.11):

$$\text{BW}_{\text{eff}} = \frac{B_{\text{move}}}{T}. \tag{2.11}$$

This follows the standard effective-bandwidth definition used for GPU kernels NVIDIA, 2026a. When parameter streaming and activation movement dominate execution, bandwidth becomes the primary runtime constraint.

Latency and throughput characterize runtime from complementary viewpoints. Latency is the time required to complete one inference request or one decoding step Sze et al., 2017 as shown in Eq.(2.12):

$$T_{\text{latency}} = T_{\text{compute}} + T_{\text{memory}} + T_{\text{overhead}}. \tag{2.12}$$

Throughput measures completed work per unit time, for example Eq.(2.13) gives requests per second or generated tokens per second Sze et al., 2017:

$$\text{Throughput} = \frac{N_{\text{outputs}}}{T}. \tag{2.13}$$

In large autoregressive models, latency and throughput often follow different scaling trends. Batching can increase throughput while single-request latency remains high. Parameter compression can reduce memory transfer while leaving runtime gains limited when execution is poorly aligned with the hardware schedule.

2.4 Conclusion

This chapter provided a detailed structural and computational analysis of the Transformer architecture. By dissecting its core components, specifically the Multi-Head Attention mechanism and the Feed-Forward Networks.We identified the exact matrices where the vast majority of the model's parameters reside. We also highlighted the inherent computational bottlenecks of this design, notably the quadratic scaling of self-attention and the massive memory bandwidth required to stream dense weight matrices during inference.

The exploration of hardware-aware efficiency metrics demonstrated that theoretical parameter reduction does not automatically translate to practical acceleration. True deployment efficiency requires a careful accounting of memory footprints, FLOPs, arithmetic intensity, and latency.

With the mathematical foundations established in Chapter 1 and the structural blueprint of the Transformer defined here, the problem of model compression can now be formulated precisely. We now know where the parameters are located, how they interact, and which metrics dictate their execution cost. The next chapter will transition from analyzing this architecture to actively modifying it, introducing Neural Network Pruning as a systematic methodology for identifying and removing non-critical weights and structures to achieve efficient deployment.

Chapter 3

Neural Network Pruning

3.1 Introduction

Within the framework established in Chapters 1 and 2, pruning can be understood as a constrained reduction of a trained network. The optimization perspective defines how pruning error is measured, the architectural structure of the network determines which components can be removed, and deployment metrics determine whether pruning leads only to nominal sparsity or to practical savings.

Pruning is an established deep neural network compression technique. Its purpose is to remove non-critical active parameters or operations in a network in a way that preserves the overall behavior of the pruned network to be similar to a reference network. In this removal process, the actual topology of the network is modified by setting weights, channels, filters, layers, or other units to zero, by masking them, or by removing them completely. The mask decision is simultaneously a compression choice and an architectural one.

The relevance of pruning follows from the over-parameterised nature of modern neural networks. Large models often contain substantial redundancy distributed unevenly across layers and parameter groups Denil et al., 2013; Michel et al., 2019. Table 3.1 illustrates the scale: each parameter requires 4 bytes at full precision (FP32) or 2 bytes at half precision (FP16), placing a hard memory floor on deployment before activations or caches are considered Devlin et al., 2019; He et al., 2016; OpenAI et al., 2023; Radford et al., 2019; Touvron et al., 2023.

Table 3.1: Parameter counts and memory footprints for selected modern models. FP32 assumes 4 bytes per parameter; FP16 assumes 2 bytes per parameter.

Model	Params	FP32 (GB)	FP16 (GB)	Notes
ResNet-50	25.6 M	0.10	0.05	Image classification
BERT-Base	110 M	0.44	0.22	Encoder model
GPT-2 (XL)	1.56 B	6.24	3.12	Autoregressive model
LLaMA-2 (7B)	6.74 B	26.9	13.5	Large decoder model
LLaMA-2 (70B)	68.9 B	275	138	Requires multiple GPUs
GPT-4	–	–	–	Parameters not publicly disclosed

A simple parameter count shows why sparsity cannot be interpreted as savings by itself.

For example, a 70B-parameter model requires about 140 GB in FP16 under dense storage, and setting half of its weights to zero without any type of physical pruning does not change this footprint unless the representation or execution pattern also changes. Prior work on sparse neural network execution shows that practical acceleration depends on whether the sparse pattern is exploitable by the software and hardware stack Cheng et al., 2024a; Gale et al., 2020. The pruning problem is therefore not only to reduce a numerical count of weights, but to choose which parameters or structures remain active under a given cost constraint. This leads us in the next section to formalize this choice using mask variables Kwon et al., 2022.

3.2 Theoretical Framework of Pruning

Pruning can be formalized as a constrained optimization problem where the goal is to find a binary mask that selects which parameters to keep while minimizing the loss increase. The mask effectively defines the pruned network by zeroing out the parameters that are removed. The optimization problem can be expressed in both constrained and regularized forms, and its solution is generally NP-hard Dong et al., 2017, leading to the use of approximate criteria based on Taylor expansions of the loss function.

3.2.1 Formal Problem Formulation

Given a parameterized network $f(\cdot;\Theta)$ whose P parameters are stored in $\Theta \in \mathbb{R}^P$, a binary mask $M \in \{0,1\}^P$ is chosen such that those parameters selected to be kept (whose mask value is 1) can be retrieved and those not (mask value 0) excluded from further calculation by computing $f(x; M \odot \Theta)$, which zeros out parameters for which $M_i = 0$ by elementwise product.

The goal is then to find the mask M and the remaining weights Θ that keep the loss $\mathscr{L}$ as low as possible while meeting a budget on how many parameters are retained:

$$\min_{\Theta, M} \mathscr{L}(M \odot \Theta) \quad \text{subject to} \quad \frac{\|M\|_0}{P} \leq \kappa, \tag{3.1}$$

where $\|M\|_0$ counts the nonzero entries of M and $\kappa \in (0,1]$ is the target density. In practice this hard constraint is often replaced by a regularised objective that avoids having to fix κ in advance:

$$\min_{\Theta, M} \mathscr{L}(M \odot \Theta) + \lambda\, \Omega(M), \tag{3.2}$$

where λ controls how aggressively compression is penalised and $\Omega(M)$ can capture parameter count, memory footprint, or latency depending on the deployment target. This regularised form folds the budget into the objective instead of enforcing it as a hard constraint (Eq. (3.2)).

Finding an optimal pruning solution is generally NP-hard, since the search space grows exponentially with the number of parameters Dong et al., 2017. Practical methods thus resort to approximate measures which evaluate how pruning a parameter or structure affects performance without considering all possible masks. Such measures can be obtained by approximating the change in training loss due to pruning, commonly through a Taylor expansion Molchanov et al., 2017.

3.2.2 Taylor Expansion Analysis of Pruning Loss

The impact of pruning on the loss can be characterised by expanding the loss around the trained weights Θ^*. Let $\delta\Theta = M \odot \Theta^* - \Theta^*$ be the perturbation induced by masking:

$$\Delta\mathscr{L} \approx \underbrace{\nabla\mathscr{L}(\Theta^*)^\top \delta\Theta}_{\text{1st order}} + \frac{1}{2}\underbrace{\delta\Theta^\top H(\Theta^*)\,\delta\Theta}_{\text{2nd order}} + \mathscr{O}(\|\delta\Theta\|^3), \tag{3.3}$$

where $H(\Theta^*) \in \mathbb{R}^{P\times P}$ is the Hessian of the loss at Θ^*. The expansion separates the pruning effect into first-order sensitivity and second-order curvature (Eq. (3.3)). The first-order term measures how sensitive the loss is to the direction of weight removal: a weight with a large gradient, when removed, shifts the loss proportionally. What the second-order term adds is curvature. It captures how steeply the gradient itself changes as weights are zeroed out, which determines whether removing a small weight from a sharp loss valley is actually safe. At a stationary point of training where $\nabla\mathscr{L}(\Theta^*) \approx 0$, the first-order term vanishes and curvature becomes the decisive quantity.

Depending on which terms are retained and how H is approximated, this expansion gives rise to the full family of saliency criteria used in practice, from magnitude scoring (zeroth order, which ignores both terms and uses weight magnitude as a proxy) through gradient-based methods (first order) to blockwise curvature methods, each of which is developed in Section 3.4 and applied to specific methods in Chapters 5 and 4.

For second-order criteria, the same Taylor expansion can be used while keeping the curvature term. This gives a more accurate estimate of the loss change, but it also requires approximating the Hessian, which is expensive at modern scales. The original OBD criterion is a simple diagonal approximation, while later methods apply the same idea blockwise to recover tractability.

3.2.3 Optimal Brain Damage and Optimal Brain Surgeon

OBD LeCun et al., 1990 is the simplest instantiation of the second-order term of the Taylor expansion: it assumes the Hessian is diagonal and applies no compensating correction to the remaining weights after the removal of a single weight, so the saliency of each weight reduces to

$$S_i^{\text{OBD}} = \frac{1}{2} H_{ii}\,\theta_i^2. \tag{3.4}$$

Under the OBD approximation, saliency becomes a Hessian-weighted square of the parameter (Eq. (3.4)). This makes ranking cheap, but it ignores interactions between weights. OBS Hassibi and Stork, 1993 drops the diagonal assumption entirely: after each removal it solves for the optimal correction of the remaining weights induced by removing a weight, achieving strictly lower loss increase than OBD. The price is the full inverse Hessian, which is intractable at modern scales. This is where the connection to modern post-training compression becomes direct: later methods recover tractability by applying OBS-style updates blockwise, one layer at

a time, with the resulting per-layer formula derived in Section 3.4.3 Frantar and Alistarh, 2023; Frantar et al., 2023.

3.2.4 Density, Sparsity, and Effective Topology

The saliency criteria above produce a score for each weight, but before deciding which weights to remove it is useful to establish what a removal decision actually means quantitatively. Given N_{act} active (nonzero) parameters out of a total of P, the *density* and *sparsity* of the pruned network are

$$\rho = \frac{N_{\text{act}}}{P}, \qquad s = 1 - \rho. \tag{3.5}$$

Density and sparsity define how many parameters remain active (Eq. (3.5)). They say nothing about where the removed weights were located. The same sparsity $s = 0.90$ can describe a network where zeros are scattered uniformly across all layers, leaving every operation dense but smaller, or one where zeros are concentrated in a few layers, leaving those layers nearly intact while others are almost entirely zeroed out, or one where the pattern is irregular enough that hardware cannot skip the zeroed operations without specialised sparse-format support Hoefler et al., 2021. A sparsity ratio is therefore a necessary but insufficient description of a pruned network; what matters equally is the *structure* of the removal pattern. This is what the next section formalises.

3.3 Pruning Structures

The pattern in which weights are removed determines whether the resulting model can be accelerated on real hardware, and how much freedom the pruning criterion has in choosing which units to remove Cheng et al., 2024a; Hoefler et al., 2021. Three structural regimes cover the main options, each sitting at a different point on the trade-off between pruning flexibility and deployment efficiency.

3.3.1 Unstructured Pruning

Unstructured pruning removes individual scalar weights without any constraint on their positions in the weight matrix Han et al., 2016. Formally, given an importance score $S(\theta_i)$ for each parameter θ_i, the binary mask $M \in \{0,1\}^P$ retains the top-κ fraction of weights by score:

$$M_i = \mathbf{1}[S(\theta_i) \geq \tau_\kappa], \tag{3.6}$$

where τ_κ is the $(1-\kappa)$ quantile of the score distribution. This connects the density constraint to a concrete mask rule: keep the weights above the score threshold and zero out the rest (Eqs. (3.1) and (3.6)).

This regime offers the most freedom because each weight is scored independently, which tends to produce the best accuracy at a given sparsity level Frankle and Carbin, 2019. The cost is deployment: the resulting zero pattern is irregular, and standard hardware cannot skip zeroed multiplications without sparse storage formats, such as compressed sparse row (CSR) formats

that store only nonzero values and their indices, and specialised kernels that skip zero-valued operations. In practice, latency reductions are modest unless sparsity exceeds roughly 90%, and even then only when paired with sparse execution support Gale et al., 2020.

3.3.2 Semi-Structured Pruning ($N:M$ Sparsity)

$N:M$ ***sparsity*** is a constrained form of unstructured pruning that imposes a regular local pattern Mishra et al., 2021. Instead of allowing arbitrary weights to be removed anywhere in the matrix, the weight vector is partitioned into consecutive non-overlapping groups $\mathscr{G}$ of M elements. Within each group, exactly N elements are retained and the remaining $M-N$ are zeroed:

$$\|m_{\mathscr{G}}\|_0 = N \qquad \forall \mathscr{G},\ |\mathscr{G}| = M, \tag{3.7}$$

Here $m_{\mathscr{G}} \in \{0,1\}^M$ is the mask restricted to group $\mathscr{G}$ and $\|\cdot\|_0$ counts its nonzero entries. The constraint forces each local group to keep the same number of weights, which makes the sparsity pattern more predictable than unconstrained unstructured pruning.

A widely used instance of semi-structured sparsity is the 2:4 pattern, where only two weights are kept in every group of four weights Cai, 2024; Y. Hu et al., 2024. This gives the hardware a predictable sparse layout based on small weight groups. NVIDIA Ampere Sparse Tensor Cores support this fine-grained sparsity pattern and can exploit it for faster matrix multiplication when the required 2:4 constraint is satisfied Mishra et al., 2021; NVIDIA Corporation, 2020.

After arbitrary weight removal and local group-wise constraints, a question that may come to mind is what happens if we try to remove coarser structures, such as complete architectural components. This is the topic of the next point.

3.3.3 Structured Pruning

Structured pruning removes entire vectors or submodules rather than individual weights, producing a strictly smaller dense model with no irregular sparsity H. Li et al., 2017. Because the result is a standard dense matrix, no sparse storage format or specialised kernel is needed; the compressed model runs on any hardware that supports ordinary matrix multiplication.

The removable unit depends on the architecture. In a convolutional network it is typically a filter or a channel; in a fully connected network it may be a neuron or a hidden dimension. The specific Transformer instantiation is covered later in Section 3.6.

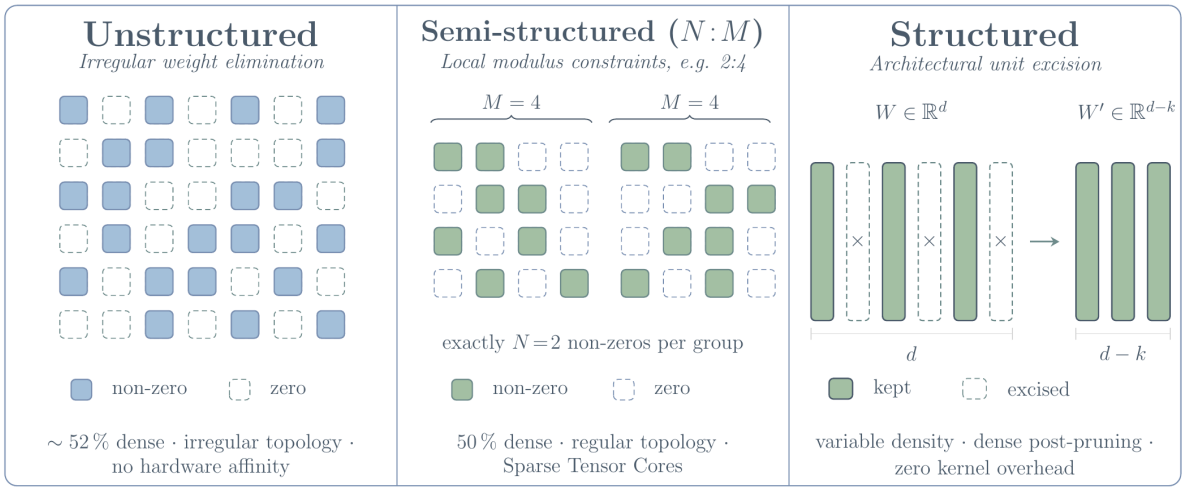
Formally, removing rows and columns from a weight matrix $W \in \mathbb{R}^{m\times n}$ using a row mask $r \in \{0,1\}^m$ and a column mask $c \in \{0,1\}^n$ yields the submatrix

$$W' = W[\mathscr{R},\mathscr{C}] \in \mathbb{R}^{m'\times n'}, \tag{3.8}$$

where $\mathscr{R} = \{i : r_i = 1\}$ and $\mathscr{C} = \{j : c_j = 1\}$ are the index sets of retained rows and columns, and $m' = |\mathscr{R}|$, $n' = |\mathscr{C}|$ are the reduced dimensions. The point of this construction is that structured pruning produces a smaller dense matrix, not a dense matrix with scattered zeros (Eq. (3.8)).

Figure 3.1 illustrates how the three regimes differ in their removal patterns. In the unstructured case, zeroed weights are scattered arbitrarily across the matrix, giving maximum freedom

but producing no contiguous blocks that hardware can skip. Semi-structured sparsity resolves this by imposing a fixed retention ratio within each local group, aligning the pattern with dedicated hardware engines. Structured pruning goes further: removing entire rows or columns leaves a fully dense submatrix with no irregular zeros and no sparse-format overhead.



Property	Unstructured	$N:M$	Structured
Sparsity ratio	Any (up to 99%+)	Fixed (e.g. 50%)	Variable
Accuracy retention	Highest	High	Lowest
HW acceleration	Custom sparse kernels	Sparse Tensor Cores	Standard dense ops
Memory format	CSR / bitmap	2:1 compressed	Dense tensor

Figure 3.1: Comparison of unstructured, semi-structured, and structured sparsity patterns in a weight matrix.

3.4 Pruning Criteria

While the sparsity structure determines the shape and hardware footprint of the compressed model, it says nothing about *which* specific weights or units should be removed. That decision is governed by the pruning criterion: given a fixed removal budget, the criterion ranks the parameters or structures by their estimated importance to the network's output, so that the least important ones are removed first.

A ***pruning criterion*** assigns an importance score $S(\theta_i)$ (or $S(g)$ for a structured group g) that serves as a proxy for the loss change $\Delta\mathscr{L}$ induced by removing that element. The ideal criterion ranks units by *true irreplaceability*: the unit with the lowest true loss impact should be removed first.

3.4.1 Magnitude-Based Criteria

The most common zeroth-order criterion evaluates a weight by its absolute value Han et al., 2015:

$$S_{\text{mag}}(w_i) = |w_i|. \tag{3.9}$$

For structured groups (e.g. a convolutional filter $W_g \in \mathbb{R}^{c_{\text{out}} \times c_{\text{in}} \times k \times k}$), the same idea is applied to the whole group rather than to one scalar weight. Common choices are the ℓ_1, ℓ_2, and ℓ_∞ norms:

$$S_g^{(1)} = \left\|W_g\right\|_1 = \sum_i |w_{g,i}|, \tag{3.10}$$

$$S_g^{(2)} = \left\|W_g\right\|_2 = \sqrt{\sum_i w_{g,i}^2}, \tag{3.11}$$

$$S_g^{(\infty)} = \left\|W_g\right\|_\infty = \max_i |w_{g,i}|. \tag{3.12}$$

These group scores extend the scalar magnitude rule to structured units: first score a whole tensor, then remove the groups with the smallest values. The ℓ_1 score measures total absolute magnitude, the ℓ_2 score measures Euclidean energy, and the ℓ_∞ score keeps only the largest entry in the group (Eqs. (3.9), (3.10), (3.11), and (3.12)).

The ℓ_2 norm is the most common because it is differentiable and rotation-invariant. The ℓ_∞ norm is sensitive to outliers and is less often used alone.

Interpretation. At a stationary point with a well-conditioned Hessian, $S_{\text{2nd}} = \frac{1}{2} H_{ii} w_i^2 \propto w_i^2$ if H_{ii} is approximately constant across parameters. Under this assumption, ranking by $|w_i|$ is equivalent to ranking by the second-order saliency LeCun et al., 1990. When Hessian diagonal entries vary greatly across layers (which they do in practice), magnitude alone may rank units from poorly-conditioned layers too aggressively.

3.4.2 Gradient-Based Criteria

First-order saliency estimates include the effect of the local gradient Molchanov et al., 2017:

$$S_{\text{grad}}(w_i) = \left| \frac{\partial \mathscr{L}}{\partial w_i} \cdot w_i \right|. \tag{3.13}$$

This is obtained from the first-order Taylor term with $\delta w_i = -w_i$ (complete removal), which turns the local gradient into a gradient-weight saliency score (Eq. (3.13)). For structured pruning, the same criterion can be extended to a removable group by summing over all constituent parameters:

$$S_{\text{grad}}(g) = \left| \sum_{i \in g} \frac{\partial \mathscr{L}}{\partial w_i} \cdot w_i \right|. \tag{3.14}$$

This produces one importance value for the whole group, not one value per scalar weight (Eq. (3.14)) Molchanov et al., 2017.

3.4.3 Second-Order Criteria

For second-order criteria, the Taylor expansion is kept to the curvature term, which gives a more accurate estimate of the loss change but requires approximating the Hessian. The original OBD criterion is a simple diagonal approximation, while later methods apply the same idea blockwise to recover tractability. There are two main approaches to second-order criteria: the diagonal approximation of OBD and the blockwise inverse Hessian of OBS.

Diagonal approximation (OBD style).

$$S_{2\text{nd}}(w_i) = \frac{1}{2} H_{ii} w_i^2. \tag{3.15}$$

The diagonal curvature term used in this score can be estimated from the Gauss-Newton approximation to the Hessian, which requires computing per-sample gradients:

$$H \approx G^\top G, \qquad G_{ij} = \frac{\partial \ell_j}{\partial w_i}, \tag{3.16}$$

where ℓ_j is the loss on sample j. The diagonal is then $H_{ii} \approx \sum_j G_{ij}^2$, giving the curvature factor used by the second-order score (Eq. (3.15)).

Blockwise inverse Hessian. The full OBS correction is too expensive to apply to all parameters at once, so modern methods apply the same idea locally inside large linear layers Frantar and Alistarh, 2023; Hassibi and Stork, 1993. In a layer with weight matrix $W \in \mathbb{R}^{m\times n}$, the calibration activations are collected in $X \in \mathbb{R}^{n\times N}$, with one column per calibration sample. These activations give the empirical input Hessian $\hat{H} = XX^\top/N$, which describes how input directions interact inside the layer. The layer is then processed column by column: when one weight is removed, the remaining weights in the current block are adjusted so that the layer output changes as little as possible.

$$W'_{:,j} = W_{:,j} - \frac{w_{:,j}}{[\hat{H}^{-1}]_{jj}} w_{:,j}^\top \hat{H}^{-1}_{j+1:,j}, \tag{3.17}$$

This correction, written in Eq. (3.17), carries the pruning error into the surviving columns instead of leaving it concentrated at the removed weight. After each removal, the remaining inverse Hessian is updated efficiently, so the next pruning decision is made with the current block state rather than the original dense one. The total cost is $O(n^3 + mn^2)$ per layer.

3.4.4 Activation-Aware Criteria

Some criteria scale saliency by the activation statistics at the weight's input. Concretely, the weight magnitude is multiplied by the ℓ_2 norm of the input feature it processes Sun et al., 2024:

$$S_{\text{act}}(W_{i,j}) = |W_{i,j}| \cdot \left\|x_j\right\|_2, \tag{3.18}$$

where $\|x_j\|_2$ is the ℓ_2 norm of the j-th input feature computed over a calibration set of N samples calculated as shown in Eq. (3.19):

$$\|x_j\|_2 = \sqrt{\sum_{s=1}^{N} x_{s,j}^2}. \tag{3.19}$$

This equals $\sqrt{N}$ times the root-mean-square (RMS) of that feature's activations. RMS measures how much energy a feature carries on average across inputs. A feature with consistently large activations has high RMS. One that is rarely triggered has low RMS and contributes little to the output regardless of its associated weight's magnitude. Equation (3.18) makes the criterion sensitive to both: a small weight on a high-RMS feature scores higher than a larger weight on a near-silent one.

Activation magnitudes are distributed unevenly across input features Sun et al., 2024; Wei et al., 2022. Magnitude-only scoring ignores this. It assigns identical importance to a weight on a high-energy feature and to an equally large weight on a near-zero feature, systematically underestimating the contribution of the first group. Scaling by $\|x_j\|_2$ corrects that bias directly.

3.4.5 Learnable and Adaptive Criteria

A fourth family introduces *trainable mask scores* $v \in \mathbb{R}^P$ that are optimised jointly with weights Louizos et al., 2018; Mallya and Lazebnik, 2018:

$$M_i = \sigma\left(\frac{v_i - \bar{v}}{\varepsilon}\right), \quad \text{or} \quad M_i = \mathbf{1}[v_i \geq \tau], \tag{3.20}$$

The mask can be kept soft during training or converted into a hard keep/remove decision (Eq. (3.20)). The discrete threshold is approximated during backpropagation by a straight-through estimator Bengio et al., 2013, which passes the gradient through the threshold as if it were the identity. The learning objective then adds a sparsity penalty so that accurate but dense masks are discouraged:

$$\mathscr{L}_{\text{sparse}} = \mathscr{L}(\Theta, M(v)) + \lambda \|v\|_1. \tag{3.21}$$

This objective balances task performance against mask sparsity (Eq. (3.21)).

These approaches are valuable especially in the setting where the training distribution is accessible and the budget for mask optimisation is not the constraint. At moderate sparsities in fine-tuning, they outperform static criteria Louizos et al., 2018; Sanh et al., 2020.

3.4.6 Criterion Comparison

Table 3.2 places the criteria of this section side by side. Read from top to bottom, it traces a single trade-off: the cheaper criteria near the top observe only the weights themselves, while the richer ones below require gradients, curvature, activations, or a training signal. That extra information is precisely what buys the accuracy gains in the last column. The table is therefore

best read as a map from *what a criterion is allowed to observe* to *how well it ranks units for removal*, which is the trade-off a practitioner balances against the available data and compute budget.

Table 3.2: Summary comparison of pruning criteria.

Criterion	Information used	Data needed	Grad?	H?	Typical accuracy
Magnitude	Weight values only	None	No	No	Baseline
Gradient (Taylor)	Gradient $\times$ weight	Train/calib	Yes	No	+1–3% over mag
OBD (§3.2.3)	Diag. curvature	Train/calib	Yes	Diag	+1–5% over mag
Blockwise OBS (§3.4.3)	Full curvature per layer	Calibration	No	Block	Best (post-train)
Activation-weighted	Magnitude $\times$ activation	Calibration	No	No	Strong post-train
Learned / trajectory mask	Score trajectory	Fine-tune	Yes	No	Best for fine-tune

3.5 Pruning Strategies

The criteria defined above state *what* to remove. A related question is *when* this removal occurs and when to let the model recover in between removals. A pruning schedule describes the pattern of mask updates and recoveries during the course of training or preparing for inference, which is what this section covers.

3.5.1 One-Shot Pruning

In ***one-shot pruning***, importance scores are computed once from the fully trained model and all removal decisions are made in a single pass Han et al., 2016. Algorithm 1 shows the procedure.

Algorithm 1 One-Shot Pruning

Input: Trained model $f(\cdot;\Theta)$, target density κ
Output: Pruned model $f(\cdot; M \odot \Theta)$
Compute $S(\theta_i)$ for all $i \in [P]$ Set threshold $\tau \leftarrow (1-\kappa)$ quantile of $\{S(\theta_i)\}$ $M_i \leftarrow \mathbf{1}[S(\theta_i) \geq \tau]$
Optional: fine-tune Θ with M fixed **return** $M \odot \Theta$

One-shot pruning requires no additional training rounds and can be applied directly to a pretrained model, which makes it simple to run. But that simplicity has a cost. All removal decisions come from a single pass over the loss landscape on a small calibration set, with no opportunity for the model to recover between mask updates. Accuracy at a given sparsity is lower than for iterative methods Han et al., 2016; Zhu and Gupta, 2018.

3.5.2 Iterative Pruning and Recovery

Iterative pruning directly addresses the limitation of one-shot pruning by interleaving mask updates with weight recovery steps Zhu and Gupta, 2018. Instead of removing all targeted weights at once, the model is pruned gradually over R rounds, with a fine-tuning phase after each round to let the remaining weights compensate for what was removed. Algorithm 2 shows a common formulation using a cubic sparsity schedule.

Algorithm 2 Iterative Pruning

Input: Trained model Θ, target density κ, rounds R
Output: Pruned model $M \odot \Theta$

$\kappa_0 \leftarrow 1.0$ `// start fully dense`
for $r = 1$ **to** R **do**
  $\kappa_r \leftarrow \text{SCHEDULE}(\kappa, r, R)$ `// interpolate density toward target`
  Compute scores $S(\theta_i)$ on current Θ `// re-evaluate importance`
  Update mask M to density κ_r `// remove lowest-scoring units`
  Fine-tune Θ with M fixed for T_r steps `// recover accuracy`
end
return $M \odot \Theta$

The SCHEDULE function can take many forms such as linear, cubic, cosine, or step, and the choice is often borrowed directly from learning rate scheduling practice, since both problems share the same goal of controlling how aggressively a quantity changes over the course of training Zhu and Gupta, 2018. Figure 3.2 illustrates several common variants.

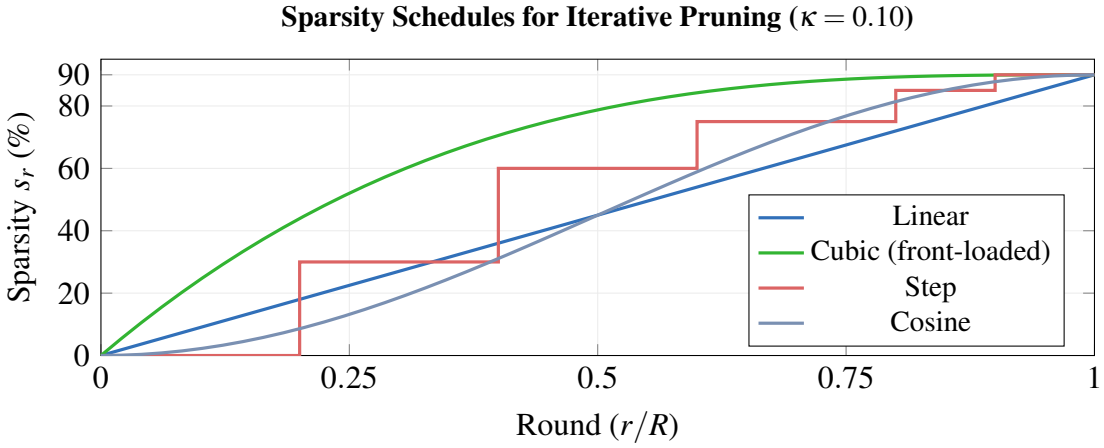


Figure 3.2: Variants of sparsity schedules

Linear growth applies equal increments each round, so the removal rate at the end of training is the same as at the start, with no settling period at high sparsity. Cubic growth concentrates most of the pruning early: by $r/R = 0.5$ roughly 87% of the target sparsity budget is already spent, and the remaining rounds operate at near-final density with small incremental changes. Step schedules skip continuous interpolation and instead apply discrete jumps at fixed intervals, holding the mask constant between updates. Cosine growth starts slowly and accelerates toward the midpoint, then decelerates as sparsity approaches the target. The last rounds under cosine resemble those of cubic. Both avoid committing large removals at high sparsity, where each pruned weight carries a larger marginal effect on the loss Zhu and Gupta, 2018.

3.5.3 Pruning During Training

A third family integrates sparsity directly into the optimisation loop by adding a regularisation term to the training loss, typically an ℓ_1 penalty of the form $\lambda \sum_i |\theta_i|$ Liu et al., 2017. This encourages weights toward zero throughout training, so that sparsity emerges gradually rather than being imposed after the fact Guo et al., 2016; Han et al., 2016. Weights that fall below a threshold are later hard-pruned. The main requirement is access to the full training distribution, which makes this approach most suitable when training from scratch or during a supervised fine-tuning stage, rather than for post-hoc compression of a frozen model.

3.5.4 Post-Training Pruning

Post-training pruning applies pruning to a frozen pretrained model using only a small calibration set, without any gradient updates to the original training loss Frantar and Alistarh, 2023. This makes it the practical default for large pretrained networks, where full retraining over the original training distribution is prohibitively expensive.

The procedure runs in four steps. First, a small calibration set of a few hundred samples is forwarded through the frozen model to collect layer-wise input activation statistics. Second, those statistics are combined with the weight values to compute a saliency score $S(\theta_i)$ for each weight or group. Third, the lowest-scoring units are zeroed out to produce the binary mask M^*. Fourth, an optional weight correction step adjusts the values of the remaining weights in each layer to compensate for the damage caused by the removals, an idea inherited from OBS and applied blockwise in modern post-training methods Frantar and Alistarh, 2023. Figure 3.3 summarises these steps.

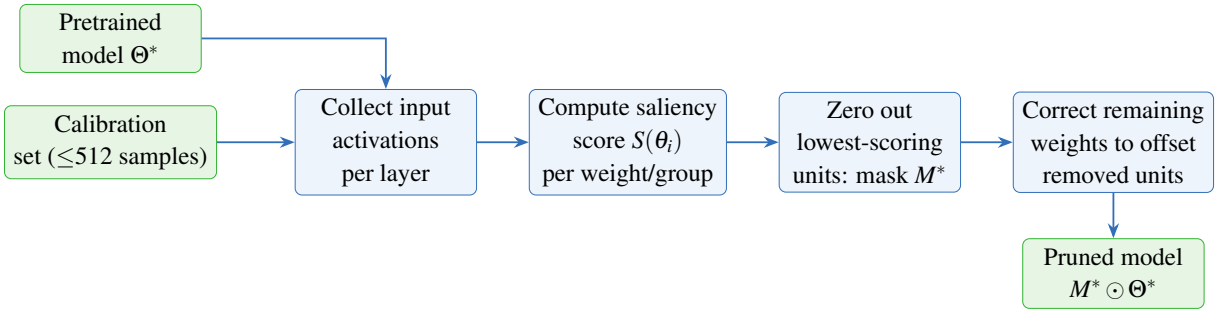


Figure 3.3: Post-training pruning pipeline.

So far, we have described pruning by means of three components: what structure should be eliminated, according to which evaluation measure, and which application strategy should be adopted. Because our specific interest is Transformers, these components now have to be identified within the architecture being compressed. In the next section, we therefore describe an instantiation of the pruning approach within Transformer models.

3.6 Pruning Transformers

The general framework described so far identifies three decisions: which units to remove, by what criterion, and according to which schedule. Applying this to a Transformer means mapping each decision to the specific computational structures of the architecture. The three main removable units are attention heads, FFN channels, and entire Transformer layers. Each carries a different cost and requires a different adaptation of the pruning formulas.

3.6.1 Attention Head Pruning

The multi-head attention module computes attention in H parallel subspaces called heads. Given an input sequence $X \in \mathbb{R}^{n \times d}$, where n is the sequence length and d is the model dimension, each head h projects X into query, key, and value spaces of dimension $d_h = d/H$ using learned matrices $W_{Q_h}, W_{K_h}, W_{V_h} \in \mathbb{R}^{d \times d_h}$, computes scaled dot-product attention, and feeds the result through the shared output projection $W_O \in \mathbb{R}^{d \times d}$.

A head mask $m \in \{0,1\}^H$ selects which of the H heads contribute to the output:

$$\mathrm{MHA}_m(X) = \mathrm{Concat}(m_1 \,\mathrm{head}_1(X), \ldots, m_H \,\mathrm{head}_H(X))\, W_O. \tag{3.22}$$

Setting $m_h = 0$ zeros out W_{Q_h}, W_{K_h}, W_{V_h}, and the corresponding block of columns in W_O (Eq. (3.22)). Since each head uses query, key, value, and output projections, the number of parameters removed per head is:

$$\Delta P_{\text{head}} = 4\, d\, d_h = \frac{4d^2}{H}. \tag{3.23}$$

Early work on head importance found that a large share of heads can be removed with no statistically significant drop in performance Michel et al., 2019; Voita et al., 2019. Heads that carry out identifiable functions, such as attending to adjacent tokens or tracking syntactic dependencies, are harder to remove, while heads whose attention patterns closely resemble those of other heads in the same layer are the most dispensable Voita et al., 2019. Importance varies considerably across layers and no single head is universally critical Michel et al., 2019.

3.6.2 FFN Channel Pruning

The feed-forward network in each Transformer layer expands the model dimension by a factor r before projecting back. For an input $X \in \mathbb{R}^{n \times d}$ and expansion ratio r, the standard FFN applies:

$$\mathrm{FFN}(X) = \phi(XW_1 + b_1)\, W_2 + b_2, \tag{3.24}$$

where $W_1 \in \mathbb{R}^{d \times rd}$, $W_2 \in \mathbb{R}^{rd \times d}$, and ϕ is a pointwise activation function. This is the standard two-projection FFN block used before channel pruning (Eq. (3.24)). A channel mask $g \in \{0,1\}^{rd}$ selects which of the rd intermediate neurons to retain:

$$\mathrm{FFN}_g(X) = \phi(XW_1 + b_1)\, \mathrm{diag}(g)\, W_2 + b_2. \tag{3.25}$$

Writing $\mathscr{K} = \{k : g_k = 1\}$ for the retained channel indices, the masked FFN can be rewritten as a dense FFN of smaller width:

$$\mathrm{FFN}_g(X) = \phi(XW_{1,\mathscr{K}} + b_{1,\mathscr{K}})W_{2,\mathscr{K},:} + b_2. \tag{3.26}$$

Removing one channel eliminates one row of W_1 and one column of W_2, which is exactly what the masked and compact forms express (Eqs. (3.25) and (3.26)). Each removed channel therefore saves $2d$ parameters. For $rd - |\mathscr{K}|$ removed channels the total saving is $(2d)(rd - |\mathscr{K}|)$.

3.6.3 Layer Pruning

Each Transformer block is an additive function: $F_l(x) = x + F_l^{(1)}(x)$, where $F_l^{(1)}(x)$ is the composition of the attention sub-layer and the FFN sub-layer. When $F_l^{(1)}(x)$ is small relative to x, the residual connection carries the bulk of the signal and the block contributes little to the representation. Removing it leaves downstream activations largely unchanged.

A natural way to measure this is to compare the magnitude of $F_l(x)$ to the magnitude of the input x over a calibration set. Using the Frobenius norm $\|\cdot\|_F$ (the square root of the sum of all squared entries of a matrix) as the measure of size, the relative block contribution is:

$$S_l^{\text{block}} = \frac{\|F_l(X)\|_F}{\|X\|_F}, \tag{3.27}$$

where the norms are computed over a batch of calibration inputs X. A block with a small relative contribution is close to an identity map, so it becomes a natural candidate for removal (Eq. (3.27)) Kurtić et al., 2023; Michel et al., 2019.

Layer pruning has the coarsest effect of any structured pruning decision since a single removal eliminates all parameters in one attention module and one FFN, a far larger parameter count than removing a head or a set of channels. This makes it effective for reducing inference latency, which scales with the number of layers, but recovery through fine-tuning is generally needed when more than a small number of layers are removed Kurtić et al., 2023.

3.7 Conclusion

The chapter formalized pruning as a constrained optimization problem, surveyed the three structural sparsity regimes, and compared the saliency criteria from magnitude scoring through second-order blockwise methods. Section 3.6 covers the Transformer instantiation.

Pruning strategy determines when the mask is updated and how much recovery the model gets between removals. One-shot, iterative, training-time, and post-training regimes each carry a different accuracy-cost profile, and the choice matters as much as the criterion. Together these components provide the vocabulary the method-level review in Chapters 4 and 5 builds on.

Chapter 4

Related work on pruning: Unstructured & Heterogeneous Methods

4.1 Introduction

Chapter 3 established the formal pruning problem, the dominant saliency criteria, and the structural regimes of sparsity. Against this theoretical background, the actual pruning literature can be read as a sequence of increasingly sophisticated responses to the same core problem: how to remove computation while preserving the behavior of the reference model under explicit deployment constraints. Because the literature is vast and varied, we divide our review of state-of-the-art methods into two chapters based on the topology of the induced sparsity. This chapter will focus on Unstructured and Heterogeneous pruning methods, while Chapter 5 will focus on Structured pruning methods. Before detailing specific algorithms, we first establish a unifying taxonomy to map the landscape of these methods.

4.2 Taxonomy of pruning methods

The main sparsity topologies were introduced formally in Section 3.3 (see Figure 3.1). From a deployment standpoint, they differ in whether pruning yields compressed storage alone or native dense-shape acceleration after model surgery.

The taxonomy adopted here follows four coupled axes. The *removable unit* determines whether the method acts on scalar weights or executable structures such as attention heads, FFN channels, and layers Kurtic et al., 2022; Michel et al., 2019. The *optimization signal* ranges from magnitude and activation statistics to differentiable mask learning, first-order Taylor criteria, second-order curvature surrogates, or search-based objectives Frantar and Alistarh, 2023; Kwon et al., 2022; Ma et al., 2023; Sun et al., 2024; Xia et al., 2022. The *recovery stage* distinguishes direct post-training pruning from methods requiring LoRA recovery, gradual fine-tuning, or distillation. The *deployment objective* determines whether the reported benefit is compressed storage, dense-shape acceleration, or measured speedup Kurtić et al., 2023; Sieberling et al., 2025; Tang et al., 2025.

Each family corresponds to a different deployment objective. Unstructured methods such as Wanda and SparseGPT primarily optimize weight-level saliency Frantar and Alistarh, 2023; Sun et al., 2024. Structured methods span differentiable mask-learning (CoFi), dependency-aware gradient methods (LLM-Pruner), Fisher-based post-training frameworks, and runtime-aware second-order methods (ZipLM) Kurtić et al., 2023; Kwon et al., 2022; Ma et al., 2023; Xia et al., 2022. Search-based methods such as EvoPress and DarwinLM treat pruning as configuration optimization under explicit budget or hardware constraints Sieberling et al., 2025; Tang et al., 2025. Figure 4.1 and Table 4.1 summarize this organization.

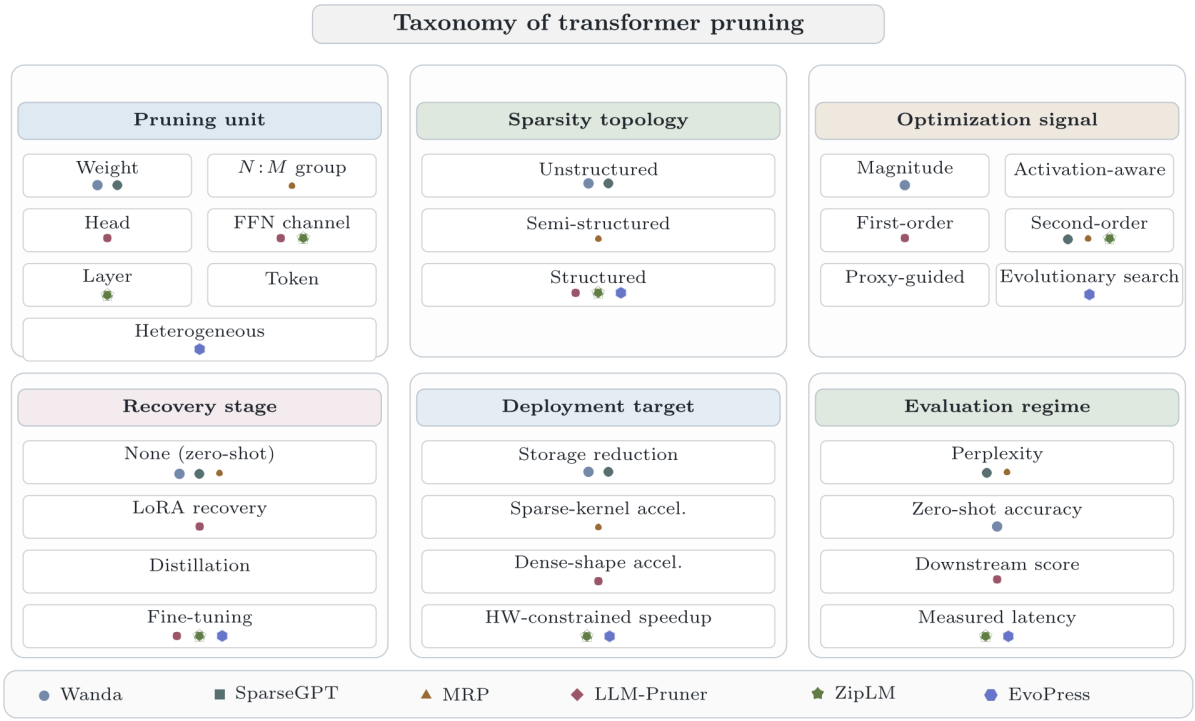


Figure 4.1: Classification of pruning methods by the unit of removal, sparsity topology, optimization signal, recovery stage, and primary deployment claim

Table 4.1: Summary of pruning methods by unit, topology, signal, recovery, and deployment claim

Method	Unit	Topology	Signal	Recovery	Primary deployment claim
Wanda	Weight	Unstructured	Activation-aware saliency	None	Storage-oriented sparsity with strong zero-shot perplexity retention
SparseGPT	Weight	Unstructured	Single-weight second-order compensation	None	High-sparsity pruning with local reconstruction
CoFi	Layers / heads / FFN / hidden dimensions	Structured	Learned hard-concrete masks with L_0 control and distillation	Full task-specific pruning and fine-tuning	Dense-shape acceleration through jointly pruned Transformer structure
LLM-Pruner	Heads / channels / blocks	Structured	First-order Taylor scoring with dependency groups	LoRA recovery	Structural compression with dense-shape execution
Fast post-training pruning	Heads / FFN filters	Structured	Fisher-based constrained mask optimization	None beyond mask tuning	FLOPs- or latency-constrained structured pruning without retraining
ZipLM	Heads / FFN dimensions	Structured	Second-order structured saliency with runtime-constrained search	Gradual fine-tuning and distillation	Measured speedup under device-specific constraints
EvoPress / DarwinLM	Layer-wise compression levels	Heterogeneous structured	Proxy-guided or evolutionary search	Search-time evaluation or proxy training	Global configuration search under budget and hardware constraints

Figure 4.1 should be read as a map of the chapter rather than as a ranking of methods. It separates the pruning literature by the kind of object being removed and by the signal used to decide removal. Table 4.1 then makes this map concrete by assigning each reviewed method to a pruning unit, topology, scoring mechanism, recovery regime, and deployment claim.

Guided by this taxonomy, the remainder of this chapter reviews methods that prioritize maximum theoretical flexibility and search space exploration: Unstructured and Heterogeneous Search-based methods (Wanda, SparseGPT, EvoPress, and DarwinLM). Structured methods are reviewed in Chapter 5.

4.3 Saliency-Based Unstructured Methods

Recall from Section 3.4 that the Taylor expansion of the loss change induced by pruning a weight θ_i produces a first-order term proportional to the gradient and a second-order term capturing curvature (Equation (3.3)). Saliency-based methods discard both terms and instead use the weight magnitude or a proxy derived from activation statistics as a surrogate for the true loss change. This zeroth-order approximation requires no gradient computation and no calibration beyond a simple forward pass, making these methods the cheapest option in the taxonomy.

Magnitude and Activation-Aware Baselines

Weight-Magnitude Pruning. Magnitude pruning uses absolute weight values as the saliency proxy:

$$\mathscr{S}_{i,j}^{\text{mag}} = |W_{i,j}|. \tag{4.1}$$

Its core assumption is that small weights contribute less to the output and can be removed with limited degradation. The method requires no gradient-based recovery and remains the simplest baseline against which more sophisticated algorithms are evaluated.

Wanda (Weights $\times$ Activations). Wanda Sun et al., 2024 refines this baseline by incorporating activation information from a small calibration set. The saliency score is

$$\mathscr{S}_{i,j}^{\text{wanda}} = |W_{i,j}| \cdot \|x_j\|_2, \tag{4.2}$$

where x_j is the j-th input activation channel and $\|x_j\|_2$ is its RMS norm estimated over N calibration samples. The key insight is that importance is jointly encoded in weight magnitude and the energy of the coupled input channel.

Both magnitude pruning and Wanda are one-shot post-training methods in the regime of Section 3.5.4: scores are computed once and no recovery training is applied afterward. Wanda processes each layer independently: it collects input activations on a small calibration set (typically 128 samples), computes per-weight saliency scores as the product of weight magnitude and activation norm, then prunes the lowest-scoring weights for each output neuron until the target sparsity p is reached. No weight update or recovery step is applied. Magnitude pruning is a pure linear scan $O(|W|)$ with no calibration; Wanda adds only an activation-collection pass, keeping the overall cost near-linear.

These saliency methods are useful because they are cheap and easy to apply, but their decisions remain local. Once pruning moves from individual weights to heads, FFN channels, or layers, the method must account for structural dependencies between parameters. This motivates gradient-based structured pruning methods, where the score is computed over groups that can be removed consistently.

4.4 Search-Based Heterogeneous Methods

Search-based methods operate over complete sparsity configurations rather than applying a single closed-form ranking rule to individual units. This is especially relevant when the quality of a pruning decision depends on how several removals interact across layers. Search-based methods are more expensive than local scoring, but they can explore a much larger space of configurations and find better-performing candidates at high compression levels. The two representative methods reviewed here are EvoPress, which performs an evolutionary search over discrete layer-wise compression profiles, and DarwinLM, which extends this search to structured pruning with training-aware selection.

EvoPress. EvoPress Sieberling et al., 2025 searches over discrete layer-wise compression profiles instead of assigning a single saliency score per layer. A candidate is a configuration $\mathbf{C} = \{c_1, \ldots, c_L\}$ where $c_\ell \in \mathscr{S}_\ell$ is the compression level for layer ℓ. Budget is preserved by level-switch mutation: one layer becomes more compressed only if another becomes less compressed, keeping the global average sparsity S_{glob} fixed. Mutations are deliberately local, with most offspring differing from the parent by only one or two switches.

Selection proceeds in multiple rounds with increasing token budgets and decreasing survivor counts:

$$\mathscr{P}^{(r+1)} = \text{Top}_{n_{r+1}}\left(\left\{\mathscr{F}_{T_r}(C) \mid C \in \mathscr{P}^{(r)}\right\}\right), \qquad T_1 < T_2 < \cdots, \; n_1 > n_2 > \cdots, \tag{4.3}$$

where the fitness signal is KL divergence to the reference model. This progressive evaluation scheme is what makes EvoPress practically efficient: early rounds use few tokens to discard poor candidates quickly, reserving the full calibration budget for finalists. The method converges in approximately 5–6 GPU hours for a standard Llama-2-7B sparsity search.

Figure 4.2 summarizes this search loop. The important point is that the algorithm evaluates complete compression profiles, selects promising candidates with increasing evaluation budgets, and mutates the retained profile while preserving the global compression constraint.

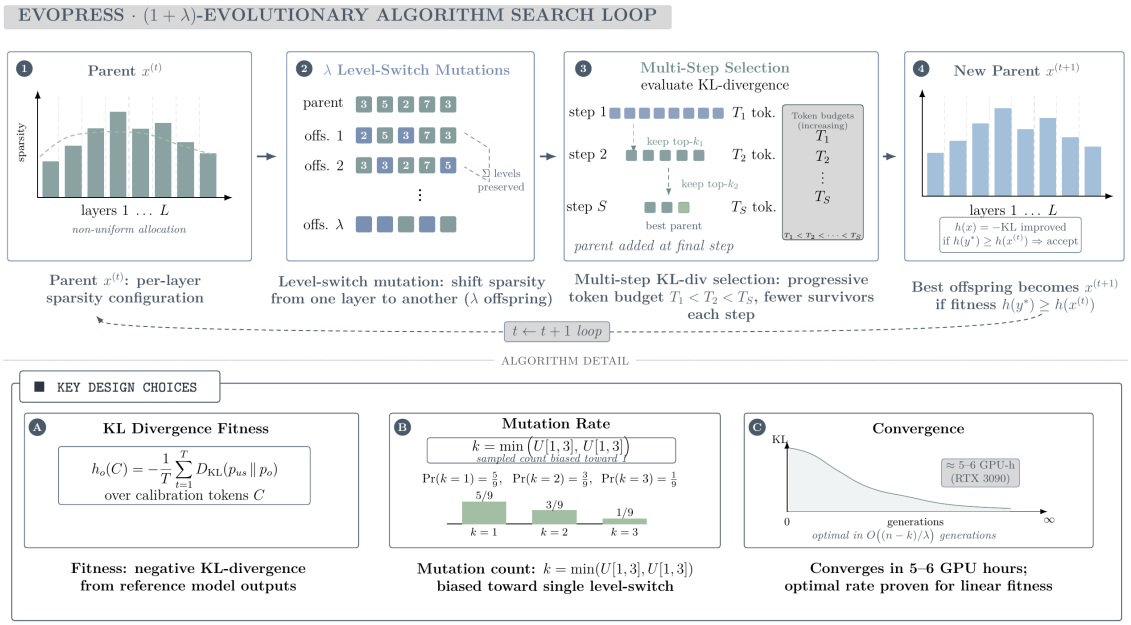


Figure 4.2: EvoPress $(1+\lambda)$ evolutionary search loop for dynamic LLM compression (Sieberling et al., 2025). The method applies budget-preserving level-switch mutation and progressively evaluates candidates with larger calibration budgets.

DarwinLM. DarwinLM Tang et al., 2025 extends evolutionary search to *structured* pruning by unifying OBS-style second-order pruning with a training-aware offspring selection strategy. It differs from EvoPress on three axes: structured granularity (heads and FFN channels), front-

loaded Hessian computation stored in a sparsity-level database (SLD), and fitness measured after a short lightweight finetune rather than from the one-shot pruned model.

For each module, DarwinLM solves the structured OBS problem at N_l equally-spaced sparsity levels and stores the pruned-and-compensated weights. The optimal structured mask and compensation are:

$$M^* = \arg\min_M \sum_{i=1}^{d_{\text{row}}} \mathbf{W}_{i,M} \big(\hat{H}^{-1}_{MM}\big)^{-1} \mathbf{W}^{\top}_{i,M}, \tag{4.4}$$

$$\delta^* = -\mathbf{W}_{:,M} \big(\hat{H}^{-1}_{MM}\big)^{-1} \hat{H}^{-1}_{M,:}, \tag{4.5}$$

where d_{row} is the output dimension of the module weight matrix, M indexes the pruned columns, and $\hat{H}$ is the empirical Hessian computed from calibration activations. During search, candidate models are assembled by looking up stored entries, avoiding any Hessian recomputation.

Each candidate receives a short lightweight finetune before evaluation:

$$\mathscr{F}(C_k) = D_{\text{KL}}\big(\hat{M} \,\|\, \text{finetune}(C_k, T_f)\big), \tag{4.6}$$

where $\hat{M}$ is the output distribution of the dense reference model and T_f is the fine-tuning token budget. Selection uses three progressive rounds ($T_f \in \{10\text{K}, 50\text{K}, 200\text{K}\}$ tokens) with decreasing survivor counts. The SLD must be computed once per model and sparsity range, making the upfront cost proportional to $L \cdot N_l$ structured OBS operations. DarwinLM compresses Llama-2-7B to 2.7B parameters with higher downstream accuracy than ShearedLlama using $5\times$ fewer post-training tokens (10B vs. 50B).

DarwinLM sits between a post-training and an iterative regime: the SLD is built offline without end-to-end training, but each search candidate receives a lightweight fine-tune before fitness evaluation. In the offline phase, the SLD is built: for each module and each of N_l sparsity levels, the structured OBS problem is solved and the pruned-and-compensated weights are stored, so that any candidate configuration can be assembled by table lookup rather than recomputation. In the search phase, candidate models are stitched together by selecting one SLD entry per module, then evaluated through that lightweight finetune. Mutation follows the same budget-preserving logic as EvoPress, swapping one module's sparsity level up only if another's is swapped down. Selection proceeds over three rounds with token budgets of 10K, 50K, and 200K and shrinking survivor counts, so weaker candidates are eliminated cheaply before the full evaluation budget is spent. The upfront SLD cost is proportional to $L \cdot N_l$ structured OBS operations and is paid once per model.

Figure 4.3 shows the corresponding DarwinLM workflow. Compared with EvoPress, the key visual distinction is the front-loaded sparsity-level database: candidate models are assembled from stored structured-pruning choices, then filtered through training-aware selection.

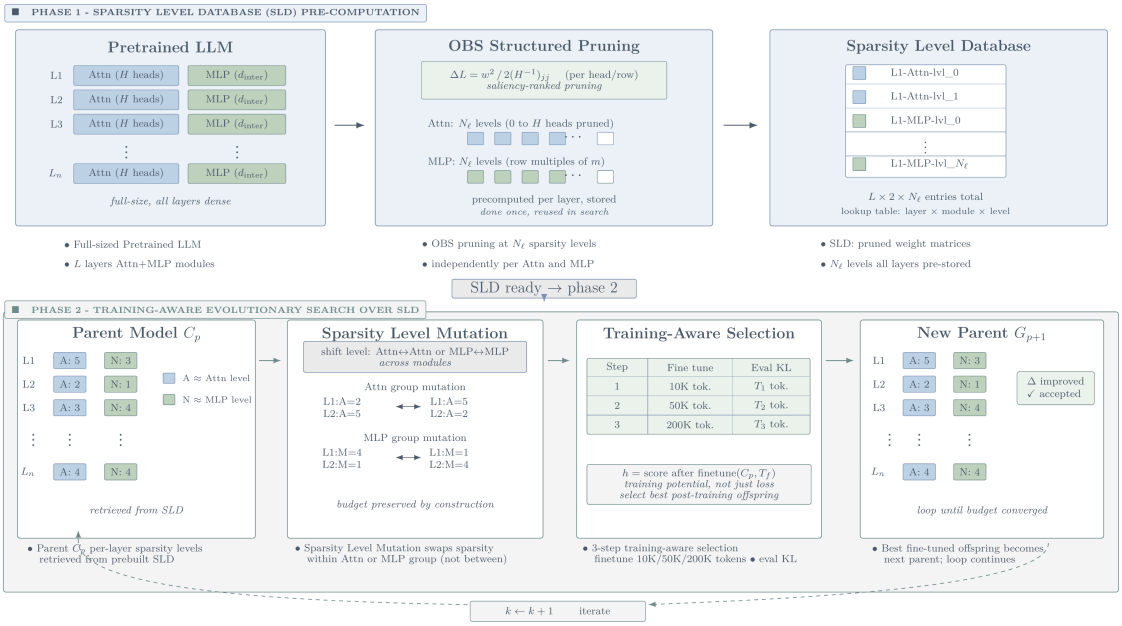


Figure 4.3: DarwinLM search loop (Tang et al., 2025). The method first builds a sparsity-level database with OBS-style structured pruning, then performs training-aware evolutionary selection over stitched candidate models.

Table 4.2 compares the practical preparation cost of the reviewed families. This table is included separately from the empirical results because a method can report strong accuracy while still requiring a substantially different calibration, recovery, benchmarking, or search budget.

Table 4.2: Preparation cost and evaluation requirements of representative pruning methods. The table separates low-cost post-training scoring, training-dependent structured pruning, curvature-based reconstruction, and population-based search.

Method	**Dominant preprocessing cost**	**Additional requirement**
Magnitude	Linear scan $O(\|W\|)$	None
Wanda	Activation collection and linear scoring	Small calibration set (≈128 samples)
CoFi	Multi-epoch joint mask and weight optimization	Task-specific teacher; ≈33 h for MNLI
LLM-Pruner	Forward/backward scoring over dependency groups	LoRA recovery (≈2.5 h on RTX 4090)
SparseGPT	Block-wise Hessian estimation and reconstruction	Calibration Hessian
MRP	Group-wise mask search and compensation	Block curvature updates for N:M groups
Fast PTPF	Diagonal or block Fisher estimation with mask tuning	Calibration set; optional latency lookup table
ZipLM	Layer-wise Hessian updates and runtime-table search	Calibration set plus hardware benchmarking pass
EvoPress / DarwinLM	Population search over compression profiles	1–6 GPU-hours; SLD pre-computation for DarwinLM

After the main method families have been described, the next step is to compare what they actually report. The empirical results below are therefore organized by pruning regime rather than by derivation, because perplexity, task accuracy, speedup, and recovery cost are not interchangeable measurements.

4.5 Conclusion

We have reviewd in the chapter the pruning methods that prioritize theoretical flexibility and search space exploration, formally known as Unstructured and Heterogeneous Search-based methods. Unstructured methods such as Wanda and SparseGPT primarily optimize weight-level saliency, while search-based methods such as EvoPress and DarwinLM treat pruning as configuration optimization under explicit budget or hardware constraints. Those methods are especially relevant for high-sparsity regimes, where the quality of pruning decisions depends on how several removals interact across layers. Search-based methods are more expensive than local scoring, but they can explore a much larger space of configurations and find better-performing candidates at high compression levels. The next chapter will review structured methods that optimize for dense-shape acceleration through joint pruning of executable structures such as attention heads, FFN channels, and layers.

Chapter 5

Related work on pruning: Structured Methods

5.1 Introduction

While Chapter 4 explored methods that remove individual weights or apply heterogeneous sparsity profiles, those approaches often rely on specialized sparse-execution hardware to achieve real-world latency reductions. This chapter shifts focus to the other side of our taxonomy: Structured Pruning.

Instead of scattering zeros throughout a weight matrix, structured pruning removes entire executable units such as attention heads, feed-forward channels, or complete Transformer layers. By excising these structural blocks, the resulting compressed model remains a standard, albeit smaller, dense neural network. This allows it to be accelerated on any standard hardware using highly optimized dense matrix multiplications, achieving immediate out-of-the-box speedups.

However, removing entire structural units is highly destructive to the model's forward pass, making the criteria for removal significantly more complex than unstructured weight magnitude. Based on the taxonomy established previously, this chapter reviews four prominent structured pruning methodologies: LLM-Pruner, CoFi, Fast Post-Training Pruning (Fast PTPF), and ZipLM. These methods represent the state-of-the-art in overcoming the destructive nature of structured removals, utilizing dependency graphs, learnable masks, and second-order optimization.

5.2 Gradient-Based Methods

Retaining the first-order term of the Taylor expansion (Equation (3.3)) gives a score proportional to the product of the gradient and the weight value. This makes the criterion sensitive to the direction of the loss landscape rather than just the weight magnitude, and it naturally extends to groups of weights by summing or aggregating the per-element scores. The methods in this section use exactly this first-order signal, applied to structured units such as heads, channels, and blocks rather than individual scalars.

Structured pruning replaces individual-weight saliency with group-level importance, because the natural units in a Transformer, namely attention heads, FFN channels, and entire layers, are coupled through residual pathways and shared projection matrices. Removing one head, for example, requires deleting consistent slices from W_Q, W_K, W_V, and W_O together; removing an FFN channel removes coordinated rows and columns from the two projection matrices. Any pruning criterion applied at this level must therefore identify groups that can be removed as a unit without breaking the forward pass Kurtic et al., 2022; Michel et al., 2019.

LLM-Pruner. LLM-Pruner Ma et al., 2023 addresses the dependency problem explicitly by constructing a graph over the neurons of the network. For neurons N_i and N_j with input set $\mathrm{In}(N)$ and output set $\mathrm{Out}(N)$, the graph encodes two rules:

$$N_j \in \mathrm{Out}(N_i) \wedge \deg^-(N_j) = 1 \implies N_j \text{ depends on } N_i, \tag{5.1}$$

$$N_i \in \mathrm{In}(N_j) \wedge \deg^+(N_i) = 1 \implies N_i \text{ depends on } N_j. \tag{5.2}$$

Here $\deg^-(N_j)$ is the in-degree of N_j (number of neurons feeding into it) and $\deg^+(N_i)$ is the out-degree of N_i. The transitive closure of these rules groups all coupled parameters into a single dependency group $\mathscr{G} = \{W_i\}_{i=1}^{M}$, which is the minimal unit that can be consistently removed.

Once groups are defined, each is scored using a first-order Taylor approximation of the loss increase over a small calibration set $\mathscr{D}$. The importance of a structure tensor W_i is estimated as

$$I_{W_i} \approx \left| \frac{\partial \mathscr{L}^\top(\mathscr{D})}{\partial W_i} W_i - \frac{1}{2} W_i^\top H W_i \right|, \tag{5.3}$$

where H is the local Hessian. The first-order gradient is retained alongside the Hessian term because the calibration set is only a proxy for the pretraining distribution; discarding it would introduce bias. An element-wise variant replaces H with a Fisher-diagonal approximation. Individual tensor scores within a group are aggregated by sum, product, max, or last-only operators, and groups with the lowest aggregate score are removed first.

LLM-Pruner sits between a post-training and a recovery regime: scoring is post-training and requires no gradient propagation through the full model, but the pruned network is then passed through a LoRA recovery stage in line with the iterative recovery approach of Section 3.5.2. LoRA adapters E. J. Hu et al., 2022 are inserted into all linear modules and fine-tuned for two epochs over roughly 50K samples. The low-rank factors are then merged back into the weight matrices, leaving no adapter overhead at inference time. The full scoring stage requires only 10 short calibration sequences, making the preprocessing cost modest relative to the recovery phase.

CoFi. CoFi Xia et al., 2022 is the main structured-pruning example of the learnable-mask criteria introduced in Section 3.4.5. It places binary masks on attention blocks, heads, FFN blocks, intermediate channels, and the shared hidden dimension, then optimises these masks

jointly with the task loss:

$$\mathrm{MHA}(X) = z_{\mathrm{MHA}} \sum_{i=1}^{N_h} z_{\mathrm{head}}^{(i)} \mathrm{Att}\left(W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}, W_O^{(i)}, X\right), \tag{5.4}$$

$$\mathrm{FFN}(X) = z_{\mathrm{FFN}} \,\mathrm{gelu}(XW_U)\,\mathrm{diag}(z_{\mathrm{int}})\,W_D, \tag{5.5}$$

where N_h is the number of attention heads, $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}, W_O^{(i)}$ are the query, key, value, and output projection matrices of head i, and W_U, W_D are the up- and down-projection matrices of the FFN. Each mask variable z is relaxed to a continuous value during training using the hard-concrete distribution for L_0 regularisation, which allows gradients to flow through the mask. The training objective combines task loss with a distillation term and a Lagrangian sparsity constraint:

$$\mathscr{L}_{\mathrm{prune}} = \mathscr{L}_{\mathrm{task}} + \mathscr{L}_{\mathrm{distil}} + \lambda_1(\hat{s} - s_0) + \lambda_2(\hat{s} - s_0)^2, \tag{5.6}$$

where s_0 is the sparsity target and $\hat{s}$ is the expected sparsity under the current mask distribution. Because the masks at different granularities interact, CoFi can discover structured subnetworks that a simple per-group ranking would miss.

CoFi is a training-time iterative method in the sense of Section 3.5.2: mask decisions and weight updates are interleaved throughout training rather than applied once. A warm-up phase first stabilizes the model weights before mask learning begins, with the sparsity constraint applied softly. The main phase then jointly optimises masks and weights under the Lagrangian objective, progressively tightening the sparsity target toward s_0 over training. Once the target is reached, each continuous mask variable is thresholded against zero to produce a binary decision, and a final fine-tuning pass is run on the surviving subnetwork. Because masks at all granularities are trained simultaneously, the discovered subnetwork reflects cross-level interactions rather than independent per-unit decisions.

Gradient-based structured pruning improves the consistency of the removed units, but the score still depends on a local approximation of how removal affects the loss. When pruning becomes more aggressive, interactions among remaining and removed weights become more important. This leads to second-order methods, which use curvature information or local reconstruction objectives to compensate for the error introduced by pruning.

5.3 Second-Order Methods

The second-order term of the Taylor expansion (Equation (3.3)) captures the curvature of the loss around the current weights. Retaining this term, rather than discarding it as saliency and gradient methods do, allows the pruning criterion to account for how much a removal actually shifts the loss given the local geometry of the loss landscape. As discussed in Section 3.4.3, this leads to the OBS framework, where the optimal weight correction after a removal is derived from the inverse Hessian. The methods in this section apply that same principle at the scale of modern LLMs, recovering tractability by restricting the Hessian computation to individual layers or column blocks.

5.3.1 Optimal Brain Compression Formulation

For a layer with weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and calibration activations $\mathbf{X} \in \mathbb{R}^{d_{\text{in}} \times n}$, the layer-wise sparse reconstruction problem is

$$\min_{\mathbf{W}^{\text{sparse}}} \|\mathbf{W}\mathbf{X} - \mathbf{W}^{\text{sparse}}\mathbf{X}\|_F^2 \quad \text{s.t.} \quad \|\mathbf{W}^{\text{sparse}}\|_0 \leq (1-p)\, d_{\text{out}} d_{\text{in}}, \tag{5.7}$$

where $\|\cdot\|_F$ is the Frobenius norm, $\|\cdot\|_0$ counts the number of nonzero entries, and $p \in (0,1)$ is the target sparsity ratio. Using the empirical Hessian proxy $\mathbf{H} = \mathbf{X}\mathbf{X}^T/n$, this becomes a curvature-aware reconstruction problem whose solution depends on $\mathbf{H}^{-1}$ to redistribute pruning error across remaining weights. The methods that follow differ in how they instantiate this objective: SparseGPT removes one scalar weight at a time, MRP removes an entire N:M group jointly, Fisher-based pruning applies curvature scoring to structured masks, and ZipLM extends the same principle to executable Transformer structures under runtime constraints.

5.3.2 Single-Weight Iterative Removal (SparseGPT)

SparseGPT Frantar and Alistarh, 2023 greedily prunes one weight at a time while applying compensation updates to remaining weights in the current block. This can be read as a blockwise version of the OBS correction introduced in Section 3.2.3 and formalised for large linear layers in Section 3.4.3. Its local objective at iteration t is

$$\min_{\delta w} \frac{1}{2} \delta w^T H \delta w \quad \text{s.t.} \quad (w + \delta w) \cdot e_t = 0, \tag{5.8}$$

where e_t is the one-hot selector for the pruned weight.

SparseGPT is a post-training one-shot method in the regime of Section 3.5.4: pruning and compensation are applied in a single forward pass over the layers without any subsequent gradient-based recovery. It processes the weight matrix in column blocks of size B. For each block, it first constructs the regularized inverse Hessian $(XX^T + \lambda I)^{-1}$. Within each block, it iteratively identifies the weight with minimum local second-order loss, sets it to zero, then updates the remaining weights in that block using the corresponding column of $\mathbf{H}^{-1}$. After processing a block, lazy batch updates are applied to adjacent blocks. The method scales as $O(d_{\text{in}}^2 d_{\text{out}} + d_{\text{in}}^3/B)$ per layer, with a calibration budget of approximately 128 samples.

It is more instructive to understand SparseGPT as a weight-level, Hessian-compensated method for pruning: each decision to prune weights is made locally, and weights are updated to make the layer output's deviation as minimal as possible Frantar and Alistarh, 2023. It is through this same lens that SparseGPT's modification for semi-structured N:M sparsity can be understood, where the compensation method is kept and mask selection is constrained to local groups. If the block size is set to $B_s = M$, and from each group of M weights the N weights with the smallest OBS reconstruction score are selected for removal, the compensation becomes more localized to a block rather than the whole layer. Within that block, however, weights are still selected and compensated one at a time, so the compensation at each step does not account for the interactions among the other weights being simultaneously removed from the same group, which is the gap MRP directly addresses.

5.3.3 Joint Optimization for Multi-Weight Removal (MRP)

The MRP framework Huang et al., 2025 generalizes second-order pruning to N:M groups. For a vectorized weight block w and binary pruning mask M, where $M_i = 1$ indicates that weight i is pruned, the objective minimizes the second-order loss increase for the full group:

$$\min_{\delta w} \frac{1}{2}\delta w^T H \delta w + g^T \delta w \quad \text{s.t.} \quad (w+\delta w)\odot M = 0, \tag{5.9}$$

where g is the gradient of the loss with respect to w, and $\odot$ denotes elementwise multiplication. Solving the group jointly yields a compensation update for the retained weights:

$$\delta w^\star = -H^{-1}_{\bar{M}\bar{M}}(g_{\bar{M}} + H_{\bar{M}M} w_M), \tag{5.10}$$

where M denotes pruned indices, $\bar{M}$ retained indices, and w_M the weight values at the pruned coordinates. This expression makes the role of the block Hessian explicit: in N:M pruning, the removed weights are not independent decisions, so compensation must be computed for the group rather than for a single scalar.

MRP is a post-training one-shot method in the regime of Section 3.5.4: no gradient updates or recovery training are applied after pruning. It operates layer by layer. For each layer, the block Hessian is estimated from calibration activations. The weight tensor is partitioned into non-overlapping groups of M consecutive weights. For each group, the joint second-order problem is solved: the mask selecting the $M-N$ weights with the highest reconstruction cost is identified, and the compensation update $\delta w^\star$ is applied to the retained N weights before the next group is processed. Because the group is treated as a unit rather than a sequence of scalar removals, the method correctly accounts for interactions among simultaneously pruned coordinates within the block.

Figure 5.1 illustrates this difference. SparseGPT compensates after removing one scalar weight, whereas MRP compensates after removing a constrained local group. The figure is therefore used to distinguish scalar second-order correction from group-level second-order correction.

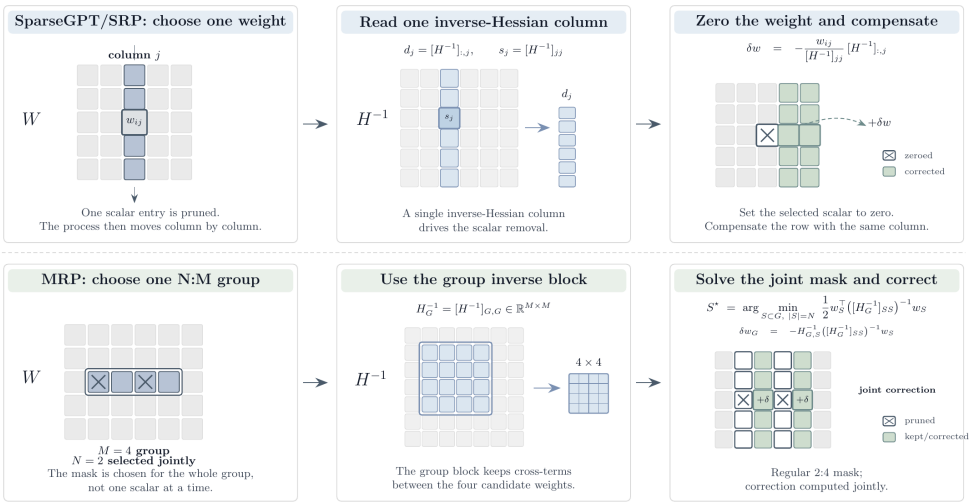


Figure 5.1: Second-order compensation strategies for unstructured and semi-structured pruning.

5.3.4 Fisher-Based Post-Training Structured Pruning

The post-training framework of Kwon et al. Kwon et al., 2022 applies second-order structured pruning without end-to-end post-pruning weight optimization. Binary masks over attention heads and FFN filters are found by solving

$$\arg\min_{\mathbf{m}} \mathscr{L}(\mathbf{m}) \quad \text{s.t.} \quad \text{Cost}(\mathbf{m}) \leq C, \tag{5.11}$$

where Cost is either a FLOPs or measured latency budget. Expanding around the dense mask and applying a diagonal Fisher approximation reduces the search to selecting units with the smallest diagonal Fisher scores subject to the cost constraint. For latency-constrained pruning, a piecewise-linear lookup table replaces the analytical cost model to incorporate real device behavior:

$$\text{LAT}(\mathbf{m}_\ell) = \begin{cases} 0, & \|\mathbf{m}_\ell\|_0 = 0, \\ c, & 0 < \|\mathbf{m}_\ell\|_0 \leq T, \\ a(\|\mathbf{m}_\ell\|_0 - T) + c, & \|\mathbf{m}_\ell\|_0 > T, \end{cases} \tag{5.12}$$

where c is the base latency incurred by any non-empty layer configuration, T is the structural threshold below which latency is approximately constant, and a is the slope capturing how measured latency grows beyond that threshold. All three constants are read from device benchmarks rather than derived analytically.

The method proceeds in four steps: (1) diagonal Fisher scores are computed for head and FFN-filter masks on a small calibration set (2K training examples); (2) a FLOPs- or latency-constrained mask search is solved under the diagonal Fisher objective; (3) a block-diagonal Fisher refinement is applied layer by layer to recover interactions ignored by the diagonal approximation; (4) surviving mask values are tuned by layer-wise linear least-squares reconstruction. Because only mask values are updated, the method avoids end-to-end weight retraining. The paper reports pruning times of 39 seconds on GLUE and 135 seconds on SQuAD, compared to 33 hours for CoFi on MNLI.

5.3.5 Inference-Aware Structured Second-Order Pruning (ZipLM)

ZipLM Kurtić et al., 2023 applies an OBS-style reconstruction objective to executable Transformer structures rather than scalar weights. For a candidate structure S with column mask $\mathbf{M}_S$, the removal score is

$$\arg\min_{S} \sum_{i=1}^{d_{\text{row}}} \mathbf{W}_{i,\mathbf{M}_S} \left(\left(\mathbf{H}^{-1} \right)_{\mathbf{M}_S,\mathbf{M}_S} \right)^{-1} \mathbf{W}_{i,\mathbf{M}_S}^T, \tag{5.13}$$

where d_{row} is the output dimension of the weight matrix and the sum aggregates the reconstruction cost across all output rows. After removing a structure, the exact compensation and Hessian update are applied through block Gaussian elimination, so that all subsequent saliency scores remain consistent with the already-pruned state.

ZipLM adds a device-level objective on top of the local second-order scoring. It benchmarks attention and FFN configurations on the target hardware and stores measured runtimes in a

lookup table. The global search then minimizes layer-wise structural sensitivity subject to a real speed constraint $\sum_\ell \tau_{\ell,s_\ell} \leq T_{\text{target}}$.

ZipLM follows an iterative pruning-and-recovery regime in the sense of Section 3.5.2: pruning steps are interleaved with fine-tuning rounds rather than applied all at once. It operates in two phases (see Figure 5.2). In the global phase, attention-head and FFN configurations are benchmarked on the target environment and a layer-wise sparsity profile satisfying the speedup requirement is selected. In the local phase, each selected layer is compressed by iteratively scoring candidate structures with the structured saliency metric, applying the optimal weight compensation, and updating the inverse Hessian by block Gaussian elimination. Recovery follows a gradual prune-and-fine-tune schedule with combined task, logit, and token-level distillation losses, at a cost of 8–10 fine-tuning epochs per pruning step.

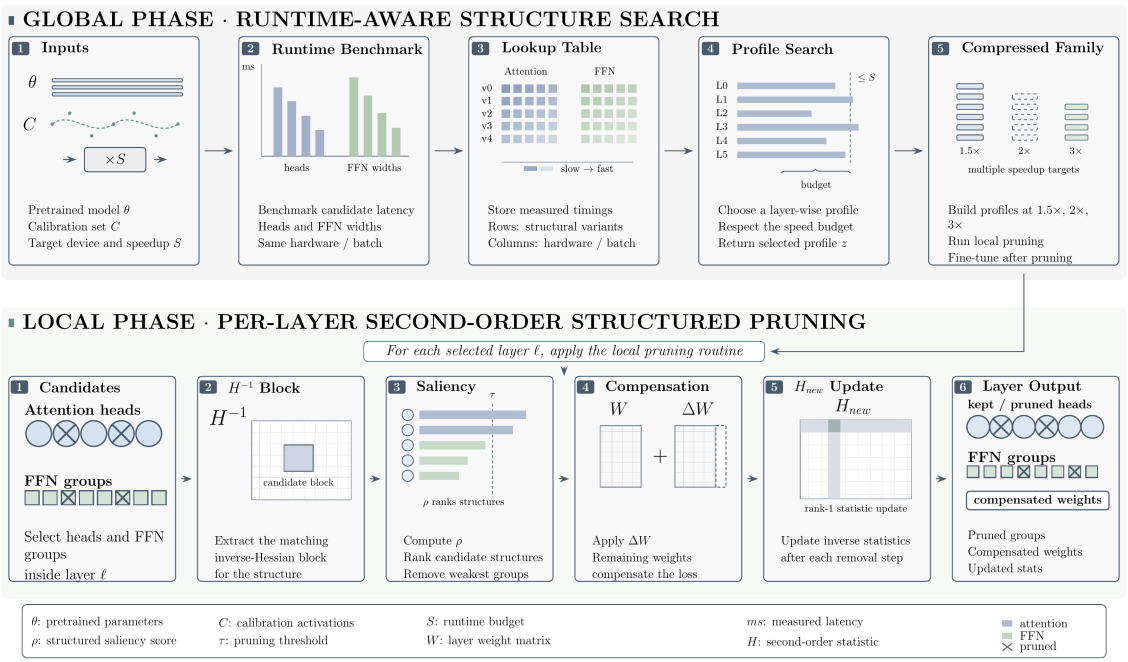


Figure 5.2: ZipLM pipeline. The global phase benchmarks attention-head and FFN configurations and searches for layer-wise sparsity profiles satisfying the required speedup. The local phase applies second-order structured pruning through inverse-Hessian scoring, compensation updates, and iterative Hessian refinement.

Second-order methods reduce local reconstruction error, but they do not fully resolve the allocation problem across the whole model. A method may know how to remove a head or channel with limited local damage and still not know how many heads or FFN channels should remain in each layer under a global budget. This is the point at which search-based methods become relevant.

5.4 Conclusion

This chapter examined structured pruning as a mechanism for generating hardware-agnostic, dense-shape acceleration. By reviewing LLM-Pruner, CoFi, Fast PTPF, and ZipLM, we illustrated how the pruning objective must adapt when removing highly coupled components like attention heads and feed-forward channels. Because structural removals are highly destructive, these methods rely on sophisticated techniques to preserve accuracy, ranging from dependency mapping to joint mask optimization and second-order Hessian compensations.

However, evaluating these methods in isolation only tells half the story. To understand the true state of Transformer compression, we must compare the structured methods discussed in this chapter with the unstructured and heterogeneous methods from Chapter 4. Therefore, the following chapter will provide a comprehensive cross-regime empirical comparison, identify the critical limitations in current evaluation protocols, and formally define the unresolved "allocation problem" that motivates the contribution of this thesis.

Chapter 6

Synthesis, Empirical Comparison, and Limitations

6.1 Introduction

The previous two chapters surveyed Transformer pruning through unstructured/heterogeneous methods (Chapter 4) and structured methods (Chapter 5). These methods optimize different units, rely on different recovery pipelines, and target different deployment metrics, so their results cannot be reduced to a single compression table. The comparison in this chapter focuses on what the evidence actually supports: where pruning remains accurate, where recovery dominates the result, and where local scoring fails to determine a strong global configuration.

6.2 Empirical Comparison

The empirical results are grouped by pruning regime because each regime reports a different kind of outcome. Unstructured and N:M methods mainly report perplexity under weight sparsity, structured methods report task accuracy or speedup after architectural removal, and search-based methods report complete pruning configurations under budget constraints.

6.2.1 Unstructured and Semi-Structured Results

Table 6.1 reports representative low, medium, and large model settings. The unstructured rows compare magnitude pruning, SparseGPT, and Wanda at 50% sparsity, and the 2:4 rows show the semi-structured setting that motivates group-aware compensation such as MRP.

Table 6.1: Representative unstructured and semi-structured pruning results on WikiText perplexity. PPL denotes perplexity; lower is better.

Model	Setting	Dense PPL	Magnitude PPL	SparseGPT PPL	Wanda PPL
LLaMA-7B	50% unstructured	5.68	17.29	7.22	7.26
LLaMA-30B	50% unstructured	4.77	7.54	5.31	5.24
LLaMA-65B	50% unstructured	3.56	5.90	4.57	4.57
OPT-13B	50% unstructured	10.13	1.2×10^4	11.17	–
OPT-175B	50% unstructured	8.35	4.3×10^4	8.21	–
OPT-13B	2:4 semi-structured	10.13	–	12.96	–
OPT-66B	2:4 semi-structured	9.34	–	10.09	–
OPT-175B	2:4 semi-structured	8.35	–	8.74	–

The results show that pure magnitude pruning can fail badly at 50% sparsity, especially on OPT models, whereas activation-aware and second-order criteria preserve perplexity much more effectively. The 2:4 rows also show that semi-structured pruning is not just unstructured pruning with a different mask shape: the local group constraint changes the compensation problem and motivates methods such as MRP.

6.2.2 Structured Pruning Results

Table 6.2 summarizes representative structured-pruning results across compression and speedup regimes. The table keeps the evaluation suite explicit: CoFi is reported on GLUE tasks and SQuADv1, Fast PTPF reports GLUE/SQuAD results, and ZipLM reports GLUE-style classification tasks and SQuADv1.

Table 6.2: Representative structured Transformer pruning results across recovery regimes. GLUE denotes the General Language Understanding Evaluation benchmark, SQuADv1 denotes Stanford Question Answering Dataset version 1.1, acc. denotes accuracy, and F1 denotes question-answering F1 score.

Method	Model	Setting	Key result
LLM-Pruner Ma et al., 2023	LLaMA-7B	20% prune ratio	94.97% zero-shot accuracy retention after LoRA recovery
	LLaMA-7B	50% prune ratio	77.4% zero-shot accuracy retention after LoRA recovery
CoFi Xia et al., 2022	BERT Base	GLUE/MNLI, $\approx 12\times$ speedup	80.6 acc.; TinyBERT_4 reports 78.8 at similar speedup
	BERT Base	SQuADv1, $\approx 8.7\times$ speedup	82.6 F1, matching TinyBERT_4 at comparable speedup
Fast PTPF Kwon et al., 2022	BERT Base	GLUE and SQuADv1, FLOPs-constrained pruning	60–70% FLOPs with $\leq 1\%$ accuracy/F1 drop
	BERT Base	GLUE and SQuADv1, latency-constrained pruning on V100	$1.34\times$–$1.47\times$ geometric-mean speedup depending on batch size
ZipLM Kurtić et al., 2023	BERT Base	GLUE/QNLI and MNLI, $2\times$ speedup	91.4 / 84.8 acc.; essentially dense-level performance
	BERT Base	GLUE/QNLI and MNLI, $10\times$ speedup	88.6 / 82.7 acc.; approximately 5M parameters
	BERT Large	SQuADv1, $10\times$ speedup	89.1 F1 with 20.9M parameters

These results are closer to deployment because the methods remove executable units, but they still depend heavily on the evaluation and recovery setup. CoFi uses task-specific training,

LLM-Pruner depends on LoRA recovery, Fast PTPF is largely post-training, and ZipLM adds device-specific runtime profiling. The numbers therefore show the achievable trade-offs within each protocol, not a clean ranking across methods.

6.2.3 Evolutionary and Search-Based Results

Table 6.3 reports representative search-based results for EvoPress and DarwinLM. EvoPress searches over depth-pruning and sparsity-allocation configurations, and DarwinLM uses structured evolutionary pruning with training-aware offspring selection.

Table 6.3: Representative search-based pruning results. Wiki2 and C4 are perplexity values where lower is better; task average is accuracy where higher is better.

Setting	Model	Budget	Method	Wiki2↓	C4↓	Task Avg.↑
Depth pruning	Mistral-7B	25%	EvoPress	**8.66**	**12.04**	–
	Mistral-7B	25%	ShortGPT	43.26	40.16	–
	Mistral-7B	25%	Shortened LLaMA	14.94	19.30	–
Depth pruning	Llama-2-7B	37.5%	EvoPress	**17.98**	**18.91**	–
	Llama-2-7B	37.5%	ShortGPT	70.94	63.51	–
	Llama-2-7B	37.5%	Shortened LLaMA	35.37	26.07	–
Unstructured allocation	Mistral-7B	70%	EvoPress	**14.42**	**16.46**	**53.8**
	Mistral-7B	70%	OWL	17.22	21.66	51.9
	Mistral-7B	70%	Uniform	23.08	30.03	49.9
Structured search	Llama-2-7B	2.7B model	DarwinLM	–	–	62.8 after 10B-token post-training
	Llama-2-7B	2.7B model	ShearedLLaMA	–	–	62.6 after 50B-token post-training

The search-based results make the allocation issue explicit. EvoPress improves over local or heuristic allocation when multiple removals interact, and DarwinLM shows that training-aware evolutionary selection can produce strong structured models with less post-training data than a fixed pruning baseline.

6.2.4 Cross-Regime Synthesis

Table 6.4 condenses these comparisons into the main implications for pruning-search design.

Table 6.4: Aggregated interpretation of the empirical pruning evidence by regime.

Regime	Representative setting	Selected evidence	Reading
Weight-level unstructured	LLaMA-7B, 50% sparsity	Dense PPL 5.68; magnitude 17.29; SparseGPT 7.22; Wanda 7.26.	Better saliency preserves perplexity, but the resulting pattern remains unstructured and does not guarantee dense-kernel speedup.
Semi-structured N:M	OPT-13B, 2:4 sparsity	Dense PPL 10.13; SparseGPT-style compensation gives 12.96 under the 2:4 constraint.	N:M sparsity needs group-aware compensation, which motivates MRP-style joint treatment of simultaneously removed weights.
Structured LLM pruning	LLM-Pruner on LLaMA-7B	20% prune ratio retains 98.6% zero-shot accuracy after LoRA recovery; 50% retains 83.6%.	Dependency-aware pruning keeps the model executable, with final quality strongly tied to recovery.
Structured encoder pruning	CoFi, Fast PTPF, and ZipLM on BERT	CoFi reaches 80.6 MNLI acc. near 12× speedup; Fast PTPF keeps 60–70% FLOPs with $\leq$1% drop; ZipLM reaches 88.6/82.7 QNLI/MNLI acc. at 10× speedup.	Structured pruning can produce executable acceleration, with protocols differing in training cost, hardware profiling, and recovery budget.
Search-based pruning	EvoPress and DarwinLM	EvoPress improves depth-pruning and sparsity-allocation baselines; DarwinLM reports 62.8 average accuracy after 10B-token post-training versus 62.6 for ShearedLLaMA after 50B tokens.	Configuration search addresses global allocation, and training-aware selection improves structured candidates beyond one-shot scoring.

6.3 Discussion

Chapters 4 and 5, together with the empirical comparison in Section 6.2, separate two questions. The first is how to rank a single removable unit, where magnitude, gradient, and curvature metrics differ. The second is how to divide the total pruning budget among the model's units, where the method families diverge more sharply. Most unresolved arguments in the literature concern this allocation question, so the discussion begins there before returning to the empirical evidence.

6.3.1 Local Scoring Versus Global Allocation

The methods reviewed in this chapter differ in implementation, pruning unit, and recovery cost, but they can all be related to the same underlying objective:

$$\min_{\mathscr{M} \in \mathscr{F}} \Delta\mathscr{L}(\mathscr{M}), \tag{6.1}$$

where $\mathscr{M}$ denotes a feasible sparsity pattern or structured subnetwork and $\Delta\mathscr{L}$ denotes the loss increase caused by pruning. The difficulty is that $\mathscr{M}$ must satisfy a compression budget while preserving the behaviour of the model.

A recurring limitation of pruning criteria is that they evaluate removable units before the final compressed model is specified. SparseGPT estimates local second-order reconstruction costs for groups of weights Frantar and Alistarh, 2023, while Wanda ranks weights using a product of weight magnitude and input activation norm Sun et al., 2024. Such criteria are effective for identifying locally removable parameters, but they do not by themselves determine

how a fixed compression budget should be distributed across layers Cheng et al., 2024b; Hoefler et al., 2021. Empirically, one-shot local pruning is effective at lower compression ratios, but iterative or globally-allocating methods prove superior at higher pruning rates Janusz et al., 2025. This gap is recognised by recent allocation-based methods: ZipLM explicitly models both local correlations within layers and global correlations across layers, and EvoPress searches over layer-wise compression profiles to minimise output distribution divergence Kurtić et al., 2023; Sieberling et al., 2025.

6.3.2 Limitations of Local Ranking at High Compression

The preceding discussion identifies a consistent pattern across the reviewed method families. Local ranking criteria can be effective at low-to-moderate compression, but their limitations become more visible as the target compression ratio increases Bai et al., 2024; Hoefler et al., 2021; Janusz et al., 2025; Sanh et al., 2020. This behaviour follows from the structure of the pruning problem: local criteria score removable units before the final compressed model is fully specified.

For Transformer models, structured pruning is naturally expressed as a layer-wise allocation problem over executable units. A pruned Transformer may retain different numbers of attention heads, FFN intermediate dimensions, hidden dimensions, or even complete MHA/FFN modules in different layers Cheng et al., 2024b; Kurtić et al., 2023; Xia et al., 2022. Thus, for a model with L layers, H attention heads per layer, and F FFN channels per layer, the retained structure can be represented by an allocation vector

$$\mathbf{s} = (h_1, f_1, h_2, f_2, \ldots, h_L, f_L),$$

subject to a global MAC budget Ω. Recent layer-wise pruning work therefore treats compression as the problem of assigning adaptive sparsity ratios across layers rather than applying a single fixed pruning rule everywhere L. Li et al., 2024.

A local ranking method instead assigns a saliency score to each removable unit and removes the lowest-scoring units until the budget is met. This implicitly approximates the global allocation problem by assuming that the contribution of each removal can be evaluated independently. EvoPress identifies this assumption explicitly in depth pruning, where block-scoring methods assume error linearity: each block is treated as contributing independently to the total compression error Sieberling et al., 2025. ZipLM motivates its structured pruning algorithm from the opposite direction, arguing that effective pruning must account for both local correlations within layers and global correlations across layers Kurtić et al., 2023.

This independence approximation is violated by the compositional structure of Transformers. Each Transformer block contains residual-connected attention and feed-forward sublayers, and self-attention layers operate on representations produced by previous layers Vaswani et al., 2017. In pruning terms, this means that modifying one layer changes the input distribution seen by later layers. SparseLLM formalises this issue by treating LLMs as chains of modules whose outputs feed subsequent modules, and argues that minimising only local layer error leads to suboptimal global pruning, especially in high-sparsity regimes Bai et al., 2024.

The same coupling appears in attention and in the global budget constraint. Scaled dot-product attention normalises query-key scores with a softmax, while multi-head attention combines several heads operating on the same layer input Vaswani et al., 2017. Empirically, attention heads have heterogeneous importance: many heads can be removed, but excessive head removal causes catastrophic drops, and sensitivity differs across attention components Michel et al., 2019. Moreover, under a fixed MAC budget, retaining more capacity in one layer requires removing capacity elsewhere. Prior work shows that non-uniform sparsity across layers is important at high compression, and DSA directly addresses this by searching for adaptive layer-wise sparsity allocation functions Gale et al., 2019; L. Li et al., 2024; Sanh et al., 2020.

6.4 Empirical Results

Having argued that allocation is more important than local scoring at high compression, the question is whether the results reflect this. On the whole, they do but only if we treat apples and oranges distinctly. The results on the unstructured model isolate the saliency rule in the absence of weight sparsity and the structured/search based readings bring back the cost of recovery and the budget allocation aspect.

6.4.1 Unstructured and Semi-Structured Methods

The unstructured pruning results allow three clean comparisons across increasing levels of local sophistication: magnitude baseline, Wanda (activation-aware saliency), and SparseGPT (second-order compensation).

The SparseGPT results primarily highlight the instability of pure magnitude pruning at high sparsity. At 50% sparsity on OPT-13B, magnitude pruning produces a perplexity of 1.2×10^4 compared to the dense model's 10.13. This indicates severe representational degradation under weight selection based solely on individual magnitude. SparseGPT reduces the same setting to 11.17, corresponding to a $+1.04$ increase over the dense model.

The comparison between Wanda and SparseGPT on LLaMA models is also instructive. At 50% sparsity on LLaMA-7B, Wanda achieves 7.26 versus SparseGPT's 7.22, a negligible 0.04 point difference. On LLaMA-13B, Wanda actually outperforms SparseGPT by 0.06 points (6.15 versus 6.21). This apparent paradox reflects two facts. First, the Wanda activation norm $\|x_j\|_2$ is an effective proxy for the weight's contribution to the output, particularly in models with large activation outliers, which LLaMA models exhibit. Second, the calibration set used for Hessian construction in SparseGPT introduces variance that at moderate sparsity levels can offset the theoretical advantage of curvature information. At higher sparsity levels (70%+), the second-order advantage of SparseGPT becomes more consistent, because local reconstruction errors accumulate and the precise compensation direction matters more.

6.4.2 Structured Methods

CoFi and ZipLM deliver similar accuracy at comparable speedup levels through substantially different preparation pipelines. CoFi reaches 80.6 on MNLI at approximately 12× speedup after 33 hours of joint mask and weight optimisation. ZipLM reaches 81.7 at 12× through a fine-tuning schedule interleaved with structured pruning passes and a dedicated hardware benchmarking stage. In both cases the reported accuracy reflects the full pipeline rather than the pruning criterion in isolation, which makes the two methods difficult to compare directly.

6.4.3 Cross-Method Comparison

The tables above support two conclusions. At 50% unstructured sparsity on OPT-13B, magnitude pruning collapses to a perplexity of 1.2×10^4 while SparseGPT reaches 11.17 (Table 6.1), showing that second-order compensation is necessary once individual weight interactions become non-negligible. At 70% unstructured sparsity on Mistral-7B, EvoPress reaches 14.42 perplexity while uniform allocation reaches 23.08 (Table 6.3), and at 37.5% depth sparsity on Llama-2-7B, EvoPress gives 17.98 against ShortGPT's 70.94, showing that global allocation search becomes the dominant factor at high compression.

6.5 Limitations of the Reviewed Methods

The comparisons so far reveal real differences between methods, yet much of the reported gap reflects how each paper was evaluated rather than how well it prunes. Three problems recur and distort almost every cross-paper claim: the evaluation regimes are inconsistent, the recovery budgets are uneven, and the results depend on a calibration set whose effect is seldom checked.

6.5.1 Evaluation Protocol Heterogeneity

A major limitation of the pruning literature is the absence of a standardised evaluation protocol. Methods are reported across at least five distinct evaluation regimes: (1) language model perplexity at fixed nominal sparsity, (2) zero-shot task accuracy after pruning with or without recovery, (3) supervised fine-tuning accuracy after pruning and recovery, (4) measured wall-clock speedup or latency on specific hardware, and (5) FLOPs reduction relative to the dense model.

These five regimes are not directly commensurable. A method reporting 99% of dense accuracy at 2× FLOPs reduction (Regime 5) addresses a different evaluation objective from a method reporting 1.5× wall-clock speedup with $<1\%$ accuracy drop (Regime 4). FLOPs constitute a hardware-agnostic theoretical measure whose relation to runtime depends on memory bandwidth, batch size, and the accelerator execution model. Some sparse execution formats require hardware-specific support, whereas standard unstructured masks often fail to yield proportional wall-clock speedup on general dense kernels.

This protocol fragmentation makes direct cross-paper comparison highly unreliable. A result table that places Wanda, CoFi, LLM-Pruner, ZipLM, and EvoPress in the same rows implicitly treats them as solutions to the same problem. In practice, these methods target different

budgets, recovery regimes, and deployment objectives. Aggregating them without qualification creates a misleading impression of a single Pareto frontier.

6.5.2 Recovery Budget Inconsistency

Closely related is the problem of inconsistent recovery budgets. Structured pruning tends to incur larger accuracy drops than unstructured pruning at equal sparsity, making the recovery stage especially consequential Xu et al., 2026. The nominal distinction between "post-training pruning" and "pruning with fine-tuning" masks a broad range of recovery costs that substantially affect reported accuracy. Within the structured methods reviewed, Fisher post-training pruning uses zero weight updates with only mask tuning and a total time of approximately 39 seconds; LLM-Pruner uses LoRA recovery on 50K samples for approximately 2.5 hours with rank-8 adapters; ZipLM uses gradual full-model fine-tuning across 8–10 epochs per pruning step with full distillation; and CoFi uses joint mask and weight optimisation across 40 epochs with a task-specific teacher.

A controlled comparison should hold the recovery budget constant rather than the nominal pruning algorithm. A CoFi model trained for 40 epochs is not directly comparable to an LLM-Pruner model with 2.5 hours of LoRA recovery. Under a matched fine-tuning budget, the observed gap could narrow substantially. The literature rarely reports such controlled comparisons, so part of the reported accuracy difference reflects recovery investment rather than pruning quality alone.

6.5.3 Calibration Set Sensitivity

All post-training pruning methods depend on a calibration set to estimate saliency scores, Hessian surrogates, or Fisher information matrices. The choice of calibration set affects the pruned model in ways that are rarely systematically studied.

For language models, the standard practice is to use 128–2048 samples from a general text corpus such as C4 or WikiText. This is a relatively small sample of the model's training distribution, and it may not adequately represent the activation statistics that determine saliency. LLM-Pruner uses only 10 short sequences for calibration, far fewer than SparseGPT's 128 samples, which increases variance in the Taylor scoring and may explain some of the variability in its zero-shot accuracy results across runs. More fundamentally, all methods that use a calibration set implicitly assume that the saliency ranking derived from calibration samples is generalisable to the full evaluation distribution. Recent work confirms that calibration data is critical to post-training pruning, especially at high sparsity, and that data more similar to the pre-training distribution yields better pruning outcomes Bandari et al., 2024; Ji et al., 2025.

6.6 Conclusion

The reviewed methods score different units, remove different structures, and require different recovery budgets, but they converge on the same limitation at high compression: local decisions

are made before the joint effect of all removals is known. Magnitude and activation-aware methods score individual weights; gradient and second-order methods improve those scores but still apply them one unit at a time; dependency-aware structured methods such as LLM-Pruner rank groups independently across layers. The empirical results show the consequence directly: local ranking degrades rapidly at aggressive depth or sparsity budgets, and search over complete configurations remains stronger.

Chapter 7

Conclusion

The survey in this report started from a simple tension in Transformer pruning. Most methods are good at scoring what looks removable, but the final compressed model depends on how the remaining capacity is distributed. A head, channel, layer, or weight group does not disappear in isolation; each change shifts the role of the rest of the network. This is why the report treats pruning as an architectural allocation problem, where the goal is to build a smaller model that can still be recovered after compression.

The proposed search follows that reading by keeping several allocation hypotheses alive under the same budget. Attention-heavy, neuron-heavy, balanced, and layer-sparse candidates are treated as different plausible answers, not as noise around one ranking. Local scores still help guide edits, but they do not decide the budget alone. A stronger pruning pipeline should keep the constraint visible, compare complete configurations, and choose recoverable allocations before spending recovery cost. The next step is to make this search cheaper and closer to deployment by adding latency, memory, and larger Transformer variants to the same allocation view.

Bibliographie

Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*. https://doi.org/10.48550/arXiv.1607.06450

Bai, G., Chai, Z., Ling, C., Wang, S., Lu, J., Zhang, N., Chen, L., Tian, D., Su, M., Wang, H., & Zhao, L. (2024). SparseLLM: Towards global pruning of pre-trained language models. *Advances in Neural Information Processing Systems*, *37*.

Bandari, A., Yin, L., Hsieh, C.-Y., Jaiswal, A. K., Chen, T., Shen, L., Krishna, R., & Liu, S. (2024). Is c4 dataset optimal for pruning? an investigation of calibration data for LLM pruning. In Y. Al-Onaizan, M. Bansal, & Y.-N. Chen (Eds.), *Proceedings of the 2024 conference on empirical methods in natural language processing* (pp. 18089–18099). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.emnlp-main.1004

Beltagy, I., Peters, M. E., & Cohan, A. (2020). Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*. https://doi.org/10.48550/arXiv.2004.05150

Bengio, Y., Léonard, N., & Courville, A. C. (2013). Estimating or propagating gradients through stochastic neurons for conditional computation. *CoRR*, *abs/1308.3432*. http://arxiv.org/abs/1308.3432

Bottou, L., Curtis, F. E., & Nocedal, J. (2018). Optimization methods for large-scale machine learning. *SIAM Review*, *60*(2), 223–311. https://doi.org/10.1137/16M1080173

Cheng, H., Zhang, M., & Shi, J. Q. (2024a). A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *46*(12), 10558–10578. https://doi.org/10.1109/TPAMI.2024.3447085

Cheng, H., Zhang, M., & Shi, J. Q. (2024b). A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *46*(12), 10558–10578. https://doi.org/10.1109/TPAMI.2024.3447085

Clevert, D.-A., Unterthiner, T., & Hochreiter, S. (2016). Fast and accurate deep network learning by exponential linear units (ELUs) [Poster]. *International Conference on Learning Representations (ICLR)*. https://doi.org/10.48550/arXiv.1511.07289

Denil, M., Shakibi, B., Dinh, L., Ranzato, M., & de Freitas, N. (2013). Predicting parameters in deep learning. *Advances in Neural Information Processing Systems*, *26*, 2148–2156. https://proceedings.neurips.cc/paper/2013/hash/7fec306d1e665bc9c748b5d2b99a6e97-Abstract.html

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In J. Burstein, C. Doran, & T. Solorio (Eds.), *Proceedings of the 2019 conference of the north american chapter of the association for computational linguistics: Human language technologies, volume 1 (long and short papers)* (pp. 4171–4186). Association for Computational Linguistics. https://doi.org/10.18653/v1/N19-1423

Dong, X., Chen, S., & Pan, S. J. (2017). Learning to prune deep neural networks via layer-wise optimal brain surgeon. *Advances in Neural Information Processing Systems*, 4857–4867. https://proceedings.neurips.cc/paper/2017/hash/c5dc3e08849bec07e33ca353de62ea04-Abstract.html

Frankle, J., & Carbin, M. (2019). The lottery ticket hypothesis: Finding sparse, trainable neural networks. *International Conference on Learning Representations*. https://openreview.net/forum?id=rJl-b3RcF7

Frantar, E., & Alistarh, D. (2023). SparseGPT: Massive language models can be accurately pruned in one-shot. In A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, & J. Scarlett (Eds.), *Proceedings of the 40th international conference on machine learning* (pp. 10323–10337, Vol. 202). PMLR. https://proceedings.mlr.press/v202/frantar23a.html

Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2023). OPTQ: Accurate post-training quantization for generative pre-trained transformers [The arXiv version is titled GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers]. *International Conference on Learning Representations*. https://openreview.net/forum?id=tcbBPnfwxS

Gale, T., Elsen, E., & Hooker, S. (2019). The state of sparsity in deep neural networks. *arXiv preprint arXiv:1902.09574*.

Gale, T., Zaharia, M., Young, C., & Elsen, E. (2020). Sparse gpu kernels for deep learning. *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–14. https://doi.org/10.1109/SC41405.2020.00021

Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In Y. W. Teh & M. Titterington (Eds.), *Proceedings of the thirteenth international conference on artificial intelligence and statistics* (pp. 249–256, Vol. 9). PMLR. https://proceedings.mlr.press/v9/glorot10a.html

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. https://doi.org/10.5555/3086952

Guo, Y., Yao, A., & Chen, Y. (2016). Dynamic network surgery for efficient dnns. *Advances in Neural Information Processing Systems*, *29*, 1379–1387. https://papers.nips.cc/paper_files/paper/2016/hash/2823f4797102ce1a1aec05359cc16dd9-Abstract.html

Han, S., Mao, H., & Dally, W. J. (2016). Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *4th International Conference on Learning Representations*. https://arxiv.org/abs/1510.00149

Han, S., Pool, J., Tran, J., & Dally, W. (2015). Learning both weights and connections for efficient neural network. *Advances in Neural Information Processing Systems*, *28*, 1135–1143. https://proceedings.neurips.cc/paper/2015/hash/ae0eb3eed39d2bcef4622b2499a05fe6-Abstract.html

Hassibi, B., & Stork, D. G. (1993). Second order derivatives for network pruning: Optimal brain surgeon. In S. J. Hanson, J. D. Cowan, & C. L. Giles (Eds.), *Advances in neural information processing systems* (pp. 164–171, Vol. 5). Morgan Kaufmann. https://proceedings.neurips.cc/paper_files/paper/1992/file/303ed4c69846ab36c2904d3ba8573050-Paper.pdf

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778. https://doi.org/10.1109/CVPR.2016.90

Hendrycks, D., & Gimpel, K. (2016). Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*. https://doi.org/10.48550/arXiv.1606.08415

Hoefler, T., Alistarh, D., Ben-Nun, T., Dryden, N., & Peste, A. (2021). Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research*, *22*(241), 1–124. http://jmlr.org/papers/v22/21-0366.html

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., … Sifre, L. (2022). Training compute-optimal large language models. *Advances in Neural Information Processing Systems*, *35*. https://doi.org/10.48550/arXiv.2203.15556

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. *International Conference on Learning Representations (ICLR)*. https://doi.org/10.48550/arXiv.2106.09685

Hu, Y., Zhao, K., Huang, W., Chen, J., & Zhu, J. (2024). Accelerating transformer pre-training with 2:4 sparsity [21–27 Jul]. In R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, & F. Berkenkamp (Eds.), *Proceedings of the 41st international conference on machine learning* (pp. 19531–19543, Vol. 235). PMLR. https://proceedings.mlr.press/v235/hu24r.html

Huang, W., Hu, Y., Jian, G., Zhu, J., & Chen, J. (2025). Pruning large language models with semi-structural adaptive sparse training. *Proceedings of the AAAI Conference on Artificial Intelligence*, *39*(23), 24167–24175. https://doi.org/10.1609/aaai.v39i23.34592

Janusz, M., Wojnar, T., Li, Y., Benini, L., & Adamczewski, K. (2025). One shot vs. iterative: Rethinking pruning strategies for model compression. *ECAI 2025: 28th European Conference on Artificial Intelligence*, *413*, 2514–2521. https://doi.org/10.3233/FAIA251100

Ji, Y., Xiang, Y., Li, J., Xia, Q., Li, P., Duan, X., Wang, Z., & Zhang, M. (2025). Beware of calibration data for pruning large language models. *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=x83w6yGIWb

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*. https://doi.org/10.48550/arXiv.2001.08361

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*. https://doi.org/10.48550/arXiv.1412.6980

Kurtic, E., Campos, D., Nguyen, T., Frantar, E., Kurtz, M., Fineran, B., Goin, M., & Alistarh, D. (2022). The optimal BERT surgeon: Scalable and accurate second-order pruning for large language models. In Y. Goldberg, Z. Kozareva, & Y. Zhang (Eds.), *Proceedings of*

the 2022 conference on empirical methods in natural language processing (pp. 4163–4181). Association for Computational Linguistics. https://doi.org/10.18653/v1/2022.emnlp-main.279

Kurtić, E., Frantar, E., & Alistarh, D. (2023). ZipLM: Inference-aware structured pruning of language models. *Advances in Neural Information Processing Systems*, *36*, 65597–65617. https://proceedings.neurips.cc/paper_files/paper/2023/hash/ced46a50befedcb884ccf0cbe8c3ad23-Abstract-Conference.html

Kwon, W., Kim, S., Mahoney, M. W., Hassoun, J., Keutzer, K., & Gholami, A. (2022). A fast post-training pruning framework for transformers. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, & A. Oh (Eds.), *Advances in neural information processing systems* (pp. 24101–24116, Vol. 35). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2022/file/987bed997ab668f91c822a09bce3ea12-Paper-Conference.pdf

LeCun, Y., Denker, J. S., & Solla, S. A. (1990). Optimal brain damage [NIPS 1989]. In D. S. Touretzky (Ed.), *Advances in neural information processing systems* (pp. 598–605, Vol. 2). Morgan Kaufmann. https://proceedings.neurips.cc/paper_files/paper/1989/hash/6c9882bbac1c7093bd25041881277658-Abstract.html

Li, H., Kadav, A., Durdanovic, I., Samet, H., & Graf, H. P. (2017). Pruning filters for efficient convnets. *International Conference on Learning Representations*. https://openreview.net/forum?id=rJqFGTslg

Li, L., Dong, P., Tang, Z., Liu, X., Wang, Q., Luo, W., Xue, W., Liu, Q., Chu, X., & Guo, Y. (2024). Discovering sparsity allocation for layer-wise pruning of large language models. *Advances in Neural Information Processing Systems*, *37*, 141292–141317. https://doi.org/10.52202/079017-4487

Liu, Z., Li, J., Shen, Z., Huang, G., Yan, S., & Zhang, C. (2017). Learning efficient convolutional networks through network slimming. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2755–2763. https://doi.org/10.1109/ICCV.2017.298

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)*. https://doi.org/10.48550/arXiv.1711.05101

Louizos, C., Welling, M., & Kingma, D. P. (2018). Learning sparse neural networks through L_0 regularization. *International Conference on Learning Representations*. https://openreview.net/forum?id=H1Y8hhg0b

Ma, X., Fang, G., & Wang, X. (2023). LLM-Pruner: On the structural pruning of large language models. *Advances in Neural Information Processing Systems*, *36*, 21702–21720. https://proceedings.neurips.cc/paper_files/paper/2023/hash/44956951349095f74492a5471128a7e0-Abstract-Conference.html

Maas, A. L., Hannun, A. Y., & Ng, A. Y. (2013). Rectifier nonlinearities improve neural network acoustic models [WDLASL 2013]. *Proceedings of the ICML Workshop on Deep Learning for Audio, Speech and Language Processing*. https://ai.stanford.edu/~amaas/papers/relu_hybrid_icml2013_final.pdf

Mallya, A., & Lazebnik, S. (2018). Packnet: Adding multiple tasks to a single network by iterative pruning. *Proceedings of the IEEE Conference on Computer Vision and Pattern*

Recognition, 7765–7773. https://openaccess.thecvf.com/content_cvpr_2018/html/Mallya_PackNet_Adding_Multiple_CVPR_2018_paper.html

McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, *5*(4), 115–133. https://doi.org/10.1007/BF02478259

Michel, P., Levy, O., & Neubig, G. (2019). Are sixteen heads really better than one? In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, & R. Garnett (Eds.), *Advances in neural information processing systems* (pp. 14014–14024, Vol. 32). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/2c601ad9d2ff9bc8b282670cdd54f69f-Paper.pdf

Mishra, A., Latorre, J. A., Pool, J., Stosic, D., Stosic, D., Venkatesh, G., Yu, C., & Micikevicius, P. (2021). Accelerating sparse deep neural networks. *arXiv preprint arXiv:2104.08378*. https://doi.org/10.48550/arXiv.2104.08378

Molchanov, D., Ashukha, A., & Vetrov, D. (2017). Variational dropout sparsifies deep neural networks. In D. Precup & Y. W. Teh (Eds.), *Proceedings of the 34th international conference on machine learning* (pp. 2498–2507, Vol. 70). PMLR. https://proceedings.mlr.press/v70/molchanov17a.html

Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted boltzmann machines. *Proceedings of the 27th International Conference on Machine Learning*, 807–814. https://www.cs.toronto.edu/~hinton/absps/reluICML.pdf

NVIDIA. (2026a). *Cuda c++ best practices guide*. Retrieved May 24, 2026, from https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html

NVIDIA. (2026b). *Matrix multiplication background user's guide*. Retrieved May 24, 2026, from https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html

NVIDIA Corporation. (2020). *NVIDIA A100 Tensor Core GPU Architecture* (White paper) (Accessed: 2026-06-01). NVIDIA Corporation. https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf

OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. (2023). GPT-4 technical report. *CoRR*, *abs/2303.08774*. https://doi.org/10.48550/arXiv.2303.08774

PyTorch Contributors. (2026a). *torch.numel*. Retrieved May 24, 2026, from https://docs.pytorch.org/docs/stable/generated/torch.numel.html

PyTorch Contributors. (2026b). *torch.Tensor.element_size*. Retrieved May 24, 2026, from https://docs.pytorch.org/docs/stable/generated/torch.Tensor.element_size.html

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language models are unsupervised multitask learners* (tech. rep.). OpenAI. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, *323*(6088), 533–536. https://doi.org/10.1038/323533a0

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *Proceedings of the 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing (EMC2) at NeurIPS 2019*. https://doi.org/10.48550/arXiv.1910.01108

Sanh, V., Wolf, T., & Rush, A. (2020). Movement pruning: Adaptive sparsity by fine-tuning. *Advances in Neural Information Processing Systems*, *33*, 20378–20389. https://proceedings.neurips.cc/paper/2020/hash/eae15aabaa768ae4a5993a8a4f4fa6e4-Abstract.html

Sieberling, O., Kuznedelev, D., Kurtic, E., & Alistarh, D. (2025). EvoPress: Accurate dynamic model compression via evolutionary search [13–19 Jul]. *Proceedings of the 42nd International Conference on Machine Learning*, *267*, 55556–55590. https://proceedings.mlr.press/v267/sieberling25a.html

Sun, M., Liu, Z., Bair, A., & Kolter, J. Z. (2024). A simple and effective pruning approach for large language models. *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=PxoFut3dWW

Sze, V., Chen, Y.-H., Yang, T.-J., & Emer, J. S. (2017). Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE*, *105*(12), 2295–2329. https://doi.org/10.1109/JPROC.2017.2761740

Tang, S., Sieberling, O., Kurtic, E., Shen, Z., & Alistarh, D. (2025). DarwinLM: Evolutionary structured pruning of large language models. *CoRR*, *abs/2502.07780*. https://doi.org/10.48550/arXiv.2502.07780

Tay, Y., Dehghani, M., Bahri, D., & Metzler, D. (2022). Efficient transformers: A survey. *ACM Computing Surveys*, *55*(6), 1–28. https://doi.org/10.1145/3530811

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. (2023). Llama 2: Open foundation and fine-tuned chat models. *CoRR*, *abs/2307.09288*. https://doi.org/10.48550/arXiv.2307.09288

Vapnik, V. N. (1998). *Statistical learning theory*. Wiley-Interscience. https://www.wiley.com/en-us/Statistical%2BLearning%2BTheory-p-9780471030034

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, *30*, 5998–6008. https://doi.org/10.5555/3295222.3295349

Voita, E., Talbot, D., Moiseev, F., Sennrich, R., & Titov, I. (2019). Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In A. Korhonen, D. Traum, & L. Màrquez (Eds.), *Proceedings of the 57th annual meeting of the association for computational linguistics* (pp. 5797–5808). Association for Computational Linguistics. https://doi.org/10.18653/v1/P19-1580

Wei, X., Zhang, Y., Zhang, X., Gong, R., Zhang, S., Zhang, Q., Yu, F., & Liu, X. (2022). Outlier suppression: Pushing the limit of low-bit transformer language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, & A. Oh (Eds.), *Advances in neural information processing systems* (pp. 17402–17414, Vol. 35). Neural Information Processing Systems Foundation. https://doi.org/10.52202/068431-1265

Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, *52*(4), 65–76. https://doi.org/10.1145/1498765.1498785

Xia, M., Zhong, Z., & Chen, D. (2022). Structured pruning learns compact and accurate models. In S. Muresan, P. Nakov, & A. Villavicencio (Eds.), *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 1513–

1528). Association for Computational Linguistics. https://doi.org/10.18653/v1/2022.acl-long.107

Xu, Z., Xu, Y., Xu, H., Liao, Y., Yao, Z., & Xie, Z. (2026). Lightweight and post-training structured pruning for on-device large language models. *IEEE Transactions on Mobile Computing*, *25*(4), 5377–5392. https://doi.org/10.1109/TMC.2025.3629756

Zhu, M., & Gupta, S. (2018). To prune, or not to prune: Exploring the efficacy of pruning for model compression. *International Conference on Learning Representations Workshop*. https://arxiv.org/abs/1710.01878

Webographie

Cai, J. (2024). Accelerating BERT with semi-structured (2:4) sparsity [Created: 2024-04-22; last updated: 2025-09-29; accessed: 2026-06-01]. https://docs.pytorch.org/tutorials/advanced/semi_structured_sparse.html